Leakage-Controlled Concept Graph Construction and Cold-Start Diagnostic Protocol for Knowledge Tracing

Tuan Dao Minh [1], Khanh-Trinh Nguyen [1], Duong Nguyen Tien [1], Quoc Khanh Ngo [3], Van-Hau Nguyen [1*], Le Hoang Son [2]

[1*]Department of Quality Assurance and Testing, Hung Yen University of Technology and Education, Hung Yen, Vietnam.
[2]Information Technology Institute, Vietnam National University, 144 Xuan Thuy Street, Cau Giay, Hanoi, Vietnam.
[3]Advanced International Research Center on Applied Artificial Intelligence, Information Technology Institute, Vietnam National University, Hanoi, Vietnam.

*Corresponding author(s). E-mail(s): nvhau66@gmail.com;
Contributing authors: tuanymc@utehy.edu.vn; trinhnk@utehy.edu.vn; duongnt@utehy.edu.vn; quockhanhngo.official@gmail.com; sonlh@vnu.edu.vn;

Abstract

Graph-enhanced knowledge tracing (KT) often infers concept graphs from the same interaction logs used for training and evaluation. When edges are pooled before the train/test split, held-out information can reach the model through the graph even if the KT optimiser sees only the training fold. We present a leakage-controlled *audit protocol*—not a new KT backbone—for knowledge-component (KC) graph construction: learner-based temporal splits, train-only prerequisite inference, DAG validation with cycle pruning, KC-level cold-start stratification, a practitioner checklist, and a DAG Disruption Rate (DDR) diagnostic. The protocol acts as a *decision-support layer* that reports which throughput×reliance regime a corpus occupies before accuracy could reveal whether a reported graph gain is trustworthy. We instantiate it on XES3G5M, ASSISTments 2012, Junyi Academy, and two synthetic logs. Audits yield acyclic train-only graphs; DDR sweeps separate augmentation operators, and downstream retraining shows

effects conditional on backbone reliance (strong DDR–AUC coupling for GKT on XES3G5M; near-zero AUC movement on graph-inert cells). On XES3G5M, under the released primary budgets (GKT: max 10 epochs, batch 4; sequence checkpoints: max 30 epochs, batch 64; Table S15), stock pyKT GKT trails *simpleKT* by about **0.041** AUC (observational; extended-training sensitivity in Section 5.3/Table S21 is not compute-matched). Graph-mediated leakage shifts headline AUC only when contamination throughput and graph reliance are both high; otherwise AUC is an unreliable leakage alarm. We position the protocol as a decision-support and leakage risk-scoring layer for graph-provenance choices rather than a route to higher headline accuracy.

Keywords: Knowledge tracing, Concept graphs, Leakage control, DAG audit, Cold-start diagnostics, Ground-truth cross-validation

1 Introduction

Knowledge tracing (KT) estimates a learner's latent mastery of knowledge components from interaction history, supporting adaptive practice, intelligent tutoring, and curriculum recommendation. The field has progressed from Bayesian Knowledge Tracing [1] to deep sequential models [2–7] and graph-enhanced models that propagate information over relations between knowledge components [8–13]. These graph-based models rely on auxiliary concept graphs, usually encoding prerequisite or similarity relations.

We use *data leakage* to mean any situation in which information that would be unavailable at deployment time influences training or evaluation—for example, when test-fold interactions support edges in an auxiliary concept graph even though the KT optimiser trains only on the training fold [14, 15]. We use *cold-start* in a KT-specific sense at the knowledge component level: a KC is cold-start when it has too few train-fold interactions to support stable parameter or neighbourhood estimates, in contrast to user- or item-level cold-start in recommender systems [16].

Current practice leaves three reproducibility gaps that motivate our audit. First, concept graphs are often inferred from the same logs used to train and evaluate KT models; pooling the full log before learner-based splitting lets held-out transitions enter the graph channel even when sequence training is fold-clean [8, 9, 17]. The broader ML literature documents leakage as a driver of irreproducible claims [14, 15]; within KT, `pyKT` and follow-on work address *sequence-level* label leakage [17, 18], but fold-specific audit of *graph provenance*—including bounded train-only sampling when logs are very large—remains under-specified relative to sequence-level label-leakage remedies [18]. Second, generic graph augmentations designed for undirected graphs can disturb directional prerequisite structure without measurement [19, 20]. Third, "cold-start" is used inconsistently in KT reporting, so aggregate AUC can mask tail-knowledge component behaviour.

Two short examples make the audit concrete. (1) **Graph-mediated leakage (worked illustration).** Suppose a transition $c_i \to c_j$ has support 40 in the train fold but support 60 once the full log is pooled, because 20 of the supporting traversals

belong to test-fold learners. Under a support-quantile cutoff the train-only count may fall below threshold and be dropped, while the pooled count clears it and the edge is retained: the graph then encodes a transition carried mainly by held-out learners the optimiser never trains on. The fraction of an edge's support that comes from held-out tokens—here 20/60—is exactly the *throughput* our audit measures ($\text{ECR}^{\text{overlap}}$, TBMR), and it is invisible to a learner-disjoint split flag. (2) **KC cold-start.** If a KC appears only a handful of times in $\mathcal{F}^f_{\text{train}}$ but frequently in $\mathcal{F}^f_{\text{test}}$, graph propagation may rely on unstable neighbourhood evidence that aggregate metrics hide—and, as Section 5.2 argues, such sparse-train KCs are precisely where a single leaked edge has the largest relative throughput.

Concretely, let $\mathcal{D} = \{(u, t, q, c, y)\}$ be a KT interaction log with learner u, timestamp t, item q, tagged knowledge component c, and correctness label $y \in \{0, 1\}$. A learner-based temporal split $\mathcal{F} = (\mathcal{F}_{\text{train}}, \mathcal{F}_{\text{val}}, \mathcal{F}_{\text{test}})$ assigns each learner to exactly one fold (we use ratios $(0.7, 0.1, 0.2)$ in all reported experiments). For evaluation fold f, the protocol must construct a fold-specific concept graph $G_f = (\mathcal{C}, E^f_{\text{pre}} \cup E^f_{\text{sim}})$ using *only* $\mathcal{F}^f_{\text{train}}$, or a representative train-only sample $\tilde{\mathcal{I}}^f_{\text{tr}} \subseteq \mathcal{F}^f_{\text{train}}$ drawn entirely within that fold when logs exceed a compute budget (Section 3.4), then train a KT model on $\mathcal{F}^f_{\text{train}}$ and score predictions on $\mathcal{F}^f_{\text{test}}$. The central question is whether reported graph-augmented KT gains remain valid when G_f is audited for leakage, acyclicity, and cold-start coverage.

Our central thesis is that graph-mediated leakage is *conditionally* harmful rather than uniformly dangerous: it shifts headline AUC only when two factors are simultaneously high—the *throughput* of held-out mass into the consumed edge set, and the *reliance* of the backbone on graph structure. Because AUC stays silent at every other configuration, it cannot serve as a leakage alarm; this is precisely why an audit layer that measures throughput *independently of accuracy* is necessary. The protocol is the instrument that tells a practitioner which throughput×reliance regime they occupy before accuracy could reveal it.

What a practitioner does differently.

Concretely, the protocol changes four decisions a team makes before reporting or deploying a graph-augmented KT model. (1) *Build the graph fold-clean:* infer $E_{\text{pre}}/E_{\text{sim}}$ only from $\mathcal{F}^f_{\text{train}}$ and export per-edge provenance, so a reported gain cannot be silently carried by held-out mass. (2) *Read the throughput indicators, not the accuracy:* inspect builder mass and TBMR (Section 3.2) to locate the corpus in the throughput×reliance grid; a high-throughput reading is a warning even when the learner-disjoint flag is silent. (3) *Gate any downstream claim on a manipulation check:* before attributing accuracy to graph structure, verify the backbone actually reads the graph (destroying it must move AUC), otherwise treat the graph as reproducibility scaffolding only. (4) *Prefer structure-preserving augmentation:* when augmenting, use the prerequisite-preserving operator that DDR rewards, which also costs the least accuracy on graph-reliant backbones (Section 4.7). These four steps are the actionable core of the checklist in Section 5.2 and the decision map in Table 16.

Figure 1 contrasts the two workflows. In workflow (a), prerequisite or similarity edges may be supported by validation/test interactions; the KT module then appears

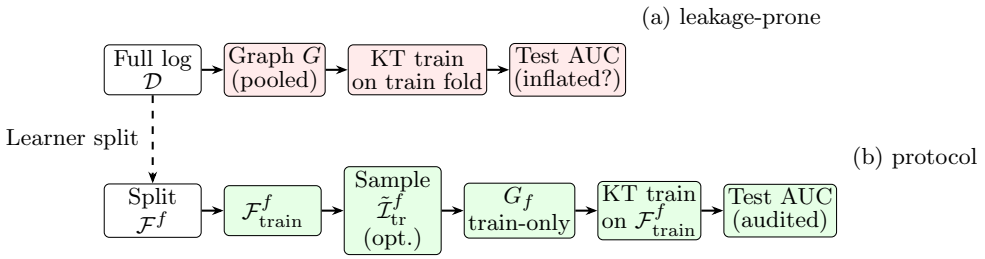


Fig. 1: Illustrative comparison of graph construction workflows. **(a)** Pooling the full log before edge inference lets held-out transitions influence G even when the KT optimiser sees only the train fold. **(b)** Our protocol builds a fold-specific graph G_f from train-only interactions. When train folds are large, edge inference may use a representative train-only sample $\tilde{\mathcal{I}}^f_{\text{tr}} \subseteq \mathcal{F}^f_{\text{train}}$ (optional; Section 3.4) to bound compute while preserving leakage control; reported benchmarks use the full train fold.

to generalise while part of the structural input already encodes future behaviour. Workflow (b) closes this channel by making graph construction a first-class, fold-conditioned step.

Leakage-controlled concept graphs matter wherever KT systems drive scheduling and gating decisions in ITS platforms, MOOC pipelines, and corporate training [21–24]. Without fold-specific audits, reported AUC improvements are difficult to attribute to modelling versus preprocessing. For applied practitioners, the deliverable is a *decision-support and risk-scoring* audit layer for *trustworthy deployment*: provenance artefacts, DAG validity checks, cold-start strata, and tiered diagnostics that help practitioners decide *whether and how* to deploy graph-augmented KT, even when headline accuracy does not move. On our primary benchmark XES3G5M, leakage-controlled evaluation separates graph backbones more clearly than on repetition-heavy corpora: under train-only E_{pre} and three-fold learner CV, the **primary** backbone trio is *simpleKT* (≈0.875), stock pyKT GKT (≈0.834; mean $\Delta \approx -0.041$, 95% CI $[-0.044, -0.038]$ in Table S16; extended-training sensitivity in Section 5.3/Table S21), and native GIKT (≈0.878). The gap is *observational* under fixed maximum epoch caps (30 for sequence checkpoints vs. 10 for graph backbones; Table S15), not a causal attribution of graph failure on XES3G5M under this protocol. SKT and DyGKT rows in Tables 8 and 9 are fork wiring sanity checks only (Supplementary Tables S1–S5 for ACC/NLL) and are excluded from primary ranking claims. A team that deploys from leaky full-log graphs (the default in original GKT/SKT pipelines [8, 9]) without provenance logs, similarity-edge feasibility checks, or cold-start strata may reach different *reporting and risk* conclusions than one that runs this protocol first—even when train-only versus full-log ablation moves headline AUC by at most 0.003 (Section 4.5).

We address the gaps above at the level of protocol rather than backbone design. We do not propose a new KT architecture. Instead, we release a leakage-controlled graph construction and audit protocol, a structural augmentation diagnostic (DDR), ground-truth cross-validation (instantiated on Junyi Academy [22, 25]), and cold-start KC reporting, evaluated on ASSISTments 2012 and XES3G5M [24] alongside two

synthetic sanity logs. Our novelty is a reproducible *audit layer* around concept-graph construction—not another sequence model [17, 26]—that separates structural disruption from downstream AUC, which graph-KT and contrastive-KT papers typically omit [8, 27].

Each contribution below answers a specific limitation (L) observed in current graph-KT practice; residual risks are stated explicitly rather than deferred to the Discussion alone.

- **C1 (addresses L1: graph leakage).** A train-only protocol for constructing and auditing KC graphs with prerequisite edges and optional similarity edges $G = (\mathcal{C}, E_{\text{pre}} \cup E_{\text{sim}})$ for KT, including split checks, edge provenance, per-relation availability reporting, and DAG validation with cycle pruning. *Residual risk:* the protocol audits declared graph provenance and tiered diagnostics; it does not certify every possible leakage path in proprietary feature pipelines.
- **C2 (addresses L2: augmentation disrupts DAGs).** The DDR metric, which quantifies how generic graph augmentations disrupt prerequisite assumptions on educational DAGs, instantiated with four formally defined label-agnostic operators and one prerequisite-preserving operator; the latter is measurably less disruptive at a matched perturbation budget, showing that DDR discriminates augmentation operators at the structural level rather than restating edge counts alone. *Downstream evidence:* under an anchored multi-seed retraining sweep, DDR predicts accuracy loss on a graph-reliant backbone that passes a manipulation check (GKT on XES3G5M: Pearson $r{\approx}0.95$ on the $p{\leq}0.3$ core, $n{=}54$; $r{\approx}0.99$ with anchors, $n{=}66$; Spearman $\rho{\approx}0.96$ on the same full pool; `prereq_preserve` costs the least AUC at matched budget), while remaining a purely structural audit on graph-inert backbones (DGEKT, and GKT on ASSISTments), where total graph destruction moves AUC by ≤ 0.003 (Section 4.7). *Residual risk:* downstream sensitivity is thus backbone- and dataset-conditional; we do not claim DDR predicts accuracy for graph-inert consumers.
- **C3 (addresses L3: cold-start reporting).** A cold-start KC diagnostic with explicit frequency-based strata and reproducible scripts on three KT benchmarks. *Residual risk:* stratum thresholds are fixed; very-cold counts can be small on real corpora and should not be over-interpreted.
- **C4 (addresses L4: expert vs. inferred graphs).** A ground-truth cross-validation procedure that quantifies, on Junyi Academy, how train-only behavioural inference of E_{pre} agrees with the expert prerequisite annotation of Chang et al. [25], using edge-level, direction-level, and reachability-level metrics across a sweep of top-K cutoffs. *Residual risk:* GT cross-validation is Junyi-only at a single expert threshold ($\theta{=}0.5$).
- **C5 (addresses L5: benchmarking under audit).** An anchored empirical observation on XES3G5M that backbone ordering under leakage-controlled graphs is *model-specific* within the primary trio (*simpleKT*, pyKT GKT, native GIKT): GKT trails *simpleKT* by mean ΔAUC ≈ -0.041 (95% CI $[-0.044, -0.038]$; Table S16) under the primary GKT maximum-10-epoch budget (batch 4), while GIKT remains competitive. A targeted extended-training check (Table S21; Section 5.3) did not isolate epoch effects under attested budgets. The trio ordering on XES3G5M

remains invariant in direction (GKT trails). Train-only versus full-log ablation ($|\Delta\text{AUC}| \leq 0.003$ and $|\Delta\text{ACC}| \leq 0.008$ on the three public benchmarks) bounds how much graph provenance alone moves headline metrics here, so C5 is a *benchmarking boundary result*, not evidence that leakage control systematically flips deployment rankings. *Residual risk:* three-fold CV limits inferential power per seed; native SKT/DyGKT runs are wiring diagnostics only.

We distill five reporting items for graph-augmented KT papers in Section 5.2 (fold provenance, cold-start strata, matched sequence baselines, ground-truth cross-validation when available, and structure-preserving versus damaging perturbation sensitivity). Supplementary Table S11 reports fold-level tests for transparency; fold-level ΔAUC magnitudes in Table 9—an order larger than fold-to-fold variation—together with Table S16 intervals characterise the primary XES3G5M trio under audit, with extended-training sensitivity consolidated in Section 5.3 and Table S21.

We assign explicit roles to the public corpora studied here. XES3G5M is **primary** (low KC-repeat, non-saturated AUC, clearest graph-backbone separation); ASSISTments 2012 is **secondary** (leakage ablation and cold-start); Junyi Academy is a **saturated sanity** corpus (ground-truth cross-validation and DAG density, not headline model ranking). Synthetic C2/C5 exercise tiny-graph audit modules. Model-comparison claims in C5 and Section 5.1 are anchored on XES3G5M unless noted. Table 8 reports cross-dataset diagnostic AUC; per-dataset ACC/NLL tables with 95% bootstrap confidence intervals are in Supplementary Tables S1–S5. All baselines use train-only exported graphs and valid+test sequence positions under three-fold learner CV. Protocol metrics (DDR, leakage tables, DAG audit) use the same split configuration; DAG audit and ground-truth cross-validation prose refer to fold 0 unless noted. Similarity edges are empty on ASSISTments (single-skill Q-matrix) and sparse on XES3G5M; Junyi is the only corpus where both E_{pre} and E_{sim} are populated under the default rule. On the three public benchmarks studied here, controlling the graph-leakage channel does not by itself yield large accuracy corrections (Section 4.5): the train-only versus full-log ablation moves headline metrics by at most 0.003 AUC and 0.008 ACC. We therefore present leakage control as an applied reproducibility and decision-support contribution together with a measured boundary result, not as a route to higher headline numbers, and we do not claim that the protocol eliminates every possible form of leakage or make claims about real-world learning outcomes.

2 Related Work

KT progressed from BKT [1] to deep sequence models (DKT, AKT, *simpleKT*) [2, 4, 5] and libraries such as `pyKT` [17, 26]. Within KT, leakage work has centred on *label leakage* in expanded KC sequences [17, 18]; the broader ML literature catalogues additional leakage types [14, 15]. The complementary channel we study—held-out information entering through the *auxiliary concept graph*—is not fixed by sequence-level remedies and remains under-audited in KT benchmarks.

Positioning against reproducibility and graph-audit practice.
KT benchmark libraries [17, 26] and label-leakage audits [18] standardise sequence splits and expanded-label hygiene; broader reproducibility work catalogues leakage modes beyond a single train/test divide [14, 15]. Our incremental contribution is not another split routine but a *fold-conditioned graph provenance layer*: exported edge lists, DAG repair logs, tiered scalar diagnostics, and practitioner reporting items that sequence-level KT fixes do not supply. Concretely, relative to `pyKT` [17] split discipline alone, the audit layer adds: (i) fold-specific exported edge artefacts (`e\_pre\_train\_only.csv`, `e\_sim\_train\_only.csv`) with per-edge provenance; (ii) logged DAG cycle pruning rather than silent repair; (iii) tiered leakage functionals plus a controlled injection stress test (Section 4.3) that shows $\mathrm{ECR}_f^{\mathrm{flag}}$ can stay zero while builder-level mass shifts; (iv) DDR structural diagnostics decoupled from AUC; (v) ground-truth cross-validation where expert graphs exist; and (vi) KC cold-start strata tied to train-fold counts. We complement—rather than replace—`pyKT`-style label audits by auditing the auxiliary graph channel those audits omit.

Graph-KT models (GKT, SKT, GIKT, DyGKT, DGEKT) [8–12] propagate mastery over KC graphs, while prerequisite-learning work infers directed relations from logs, content, or experts [25, 28, 29]. Graph-KT papers typically treat the graph as given and optimise predictive loss. Original GKT and SKT pipelines pool transition statistics over the full log before learner splitting [8, 9]; our protocol isolates that provenance choice. Expert annotations (e.g. Junyi) and behavioural inference capture related but non-identical structure; we quantify disagreement rather than treating one as a proxy for the other.

Graph SSL and contrastive KT import label-agnostic augmentations [19, 27, 30], but educational prerequisite DAGs impose directionality that standard drops can violate. KT reproducibility has focused on code and splits [17] rather than graph audit logs. Our work is orthogonal to new KT architectures: it supplies an audit layer—leakage diagnostics, DAG pruning logs, DDR, ground-truth cross-validation, and cold-start strata—that graph-KT and contrastive-KT papers typically omit.

3 Protocol

Let $\mathcal{D}$ be a KT interaction log, $\mathcal{C}$ the set of knowledge components, and $\mathcal{F} = (\mathcal{F}_{\text{train}}, \mathcal{F}_{\text{val}}, \mathcal{F}_{\text{test}})$ a learner-based temporal split. The protocol constructs a fold-specific graph $G_f = (\mathcal{C}, E^f_{\text{pre}} \cup E^f_{\text{sim}})$ using only $\mathcal{F}^f_{\text{train}}$. Figure 2 summarises module dependencies; Supplementary Figures S4–S5 give artefact-level data flow and prerequisite-edge construction (Section B).

Figure 2 summarises module dependencies. Split discipline is a hard precondition: without learner-disjoint folds, downstream diagnostics are not interpretable. Graph construction and DAG audit are coupled: train-only inference can introduce cycles, so pruning must be logged before any graph-augmented KT run. Leakage metrics, DDR, cold-start strata, and ground-truth cross-validation are *parallel* audits—they do not retrain models—which keeps the protocol lightweight but means it cannot certify every possible leakage path (e.g. hidden features in proprietary pipelines). Supplementary

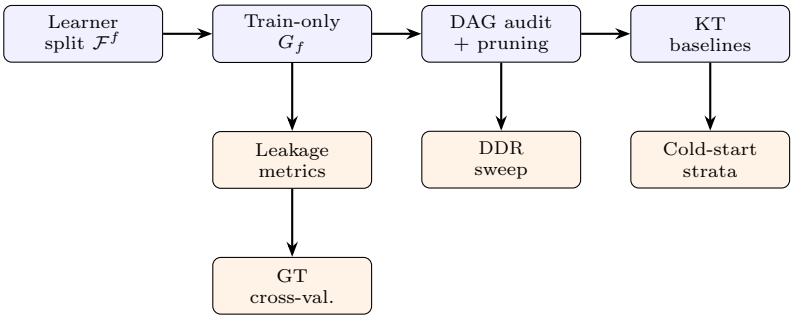


Fig. 2: Research framework connecting protocol modules (Section 3) and experiment tracks (Section 4). Fold-conditioned provenance, DAG repair, structural diagnostics decoupled from AUC, and optional expert cross-validation are the core audit outputs.

Figure S4 (Figure 3) gives artefact-level data flow from raw logs through cold-start stratum reporting.

3.1 Formal Problem Statement and Leakage Audit

For each evaluation fold f, the protocol seeks a graph $G_f = (\mathcal{C}, \mathcal{E}_f)$ with $\mathcal{E}_f = E^f_{\text{pre}} \cup E^f_{\text{sim}}$ that is *feasible* with respect to train-only evidence and structural constraints, then verifies low direct leakage on scalar diagnostics. Let $\mathcal{I}^f_{\text{tr}}$ denote train-fold interactions and $\mathcal{H}_f = \mathcal{F}^f_{\text{val}} \cup \mathcal{F}^f_{\text{test}}$ held-out interactions.

Decision variables.

Edge sets $E^f_{\text{pre}} \subseteq \mathcal{C} \times \mathcal{C}$ and $E^f_{\text{sim}} \subseteq \mathcal{C} \times \mathcal{C}$, edge weights $w_{ij} \geq 0$, and pruning decisions on cyclic subsets of E^f_{pre}.

Design intent (not a deployed solver).

The released code does *not* solve a joint numerical graph programme. Prerequisite and similarity edges are inferred in separate greedy pipelines with λ=0 in practice; sparsity enters through support quantiles, top-k per-node caps, and a global edge budget K (Section 3.4), followed by deterministic cycle pruning. The conceptual objective (Eq. (8)) and feasibility constraints (C1)–(C4) are stated formally in Supplementary Section A; treat them as design documentation, not as an optimiser executed end-to-end.

Constraints (summary).

C1–C2 block cross-fold and cross-learner contamination; C3–C4 encode the DAG audit and top-K/top-k filters of Section 3.4. We deliberately do *not* impose TBMR_f=0 (Eq. (4)) as a feasibility constraint: temporal-transition inference necessarily draws evidence from each learner's full train timeline, so a zero rate would be attainable only by discarding late-train evidence. TBMR_f is instead reported as a within-train stress

diagnostic in the sanity/stress tier below; the released runs accordingly show non-zero values on dense corpora (Table 5).

Audit functionals (diagnostics, not training losses).

Because C1 cannot always be verified from AUC alone, the protocol reports four scalar functionals on G_f: $\mathrm{ECR}_f^{\mathrm{flag}}$ (Eq. (1)), $\mathrm{ECR}_f^{\mathrm{overlap}}$ (Eq. (2)), $|\rho_f|$ (Eq. (3)), and TBMR_f (Eq. (4)). We treat them in three tiers: **primary pass/fail**—fold-specific provenance logs, $\mathrm{ECR}_f^{\mathrm{flag}}$=0, and (when available) ground-truth cross-validation; **empirical bounds**—train-only versus full-log ΔAUC/ACC and cold-start strata; **sanity/stress**— $\mathrm{ECR}_f^{\mathrm{overlap}}$, $|\rho_f|$, and TBMR_f. Under learner-disjoint splits, $\mathrm{ECR}_f^{\mathrm{flag}}$=0 is expected and does not by itself certify graph quality; near-unity overlap can occur when curricula repeat across disjoint learners—a binary indicator that masks how *concentrated* held-out mass is on each edge. We therefore also report the distribution of per-edge held-out shares $\pi_e = n_{\mathrm{hi}}(e)/(n_{\mathrm{tr}}(e) + n_{\mathrm{hi}}(e))$ (Table 5, columns $> 50\%$ and p90): even when $\mathrm{ECR}_f^{\mathrm{overlap}} \approx 1$, most edges keep $\pi_e \ll 1$ on dense logs, whereas tail-knowledge component neighbourhoods—the same locus as the cold-start strata in Section 3.7—are where a few edges can reach $\pi_e > 0.5$. TBMR_f quantifies *within-train* temporal mixing (Section 3.2), not cross-split leakage. High AUC does not certify absence of leakage; the tiered diagnostics complement discrimination scores.

3.2 Leakage Control

As introduced in Section 1, leakage here means any information path by which held-out labels, held-out interactions, future timestamps, learner identities, or graph evidence influence graph construction, feature construction, model selection, or evaluation although such information would not be legitimately available at the corresponding prediction point. Section 1 gives the intuitive definition; here we specialise it to *edge provenance* in KC graphs (Table 1), following the learn–predict separation view in data mining [14, 15] and complementing sequence-level label leakage in KT [17, 18].

We audit five leakage classes at the level of edge provenance (Table 1): structural, edge, target, temporal, and cold-start-neighbourhood classes cover learner pooling, held-out transition support, outcome-conditioned graph artefacts, boundary violations, and tail-knowledge component edges dominated by non-train mass.

Each inferred edge is logged with its fold of origin, timestamp range, relation type, and train-only flag; severe violations halt the pipeline. In addition, we report four scalar *direct* leakage diagnostics that do not reduce to predictive accuracy. Let $\mathcal{U}_f^{\mathrm{tr}}$, $\mathcal{U}_f^{\mathrm{va}}$, and $\mathcal{U}_f^{\mathrm{te}}$ denote the learner-id sets in train, validation, and test under fold f. The learner overlap flag is

$$\mathrm{ECR}_f^{\mathrm{flag}} := \mathbb{1}\left[(\mathcal{U}_f^{\mathrm{tr}} \cap \mathcal{U}_f^{\mathrm{va}}) \cup (\mathcal{U}_f^{\mathrm{tr}} \cap \mathcal{U}_f^{\mathrm{te}}) \cup (\mathcal{U}_f^{\mathrm{va}} \cap \mathcal{U}_f^{\mathrm{te}}) \neq \emptyset\right]. \tag{1}$$

When folds are learner-disjoint, $\mathrm{ECR}_f^{\mathrm{flag}} = 0$ for every fold—the regime enforced below.

Let $\mathcal{E}_f$ denote the graph edge set (prerequisite and similarity) consumed when evaluating fold f, and let $\mathcal{H}_f = \mathcal{F}_{\mathrm{val}}^f \cup \mathcal{F}_{\mathrm{test}}^f$ denote held-out interactions under that split. For edge e, let $S_{\mathrm{hi}}(e)$ be held-out interaction tokens counted toward e's evidence

Table 1: Taxonomy of leakage classes audited by the protocol: typical source of contamination, a concrete KT example, and the primary direct diagnostic used in this paper (Section 3.2).

Class	Source	Example	Metric
Structural	Learner identifiers reused across train, validation, and test folds	The same learner contributes interactions to both train and held-out partitions under fold f	$\mathrm{ECR}^{\mathrm{flag}}$
Edge	Interactions outside $\mathcal{F}^f_{\mathrm{train}}$ used as evidence for edges	Prerequisite or similarity edge retained because val/test transitions inflate co-occurrence or transition counts	$\mathrm{ECR}^{\mathrm{overlap}}$
Target	Outcomes or mappings pooled across splits	Q-matrix or calibration statistics fit on all learners; edge weights implicitly tuned against aggregated correctness	$\lvert\rho\rvert$
Temporal	Violating the timestamp boundary τ_u per learner u	Post-boundary attempts included when counting $c_t \to c_{t+1}$ support or when building time-sensitive features	TBMR
Cold-start neighbourhood	Tail-knowledge component edges dominated by non-train support	Rare-knowledge component similarity edge driven mainly by interactions that belong to val/test under the split	$\mathrm{ECR}^{\mathrm{overlap}}$ (+ strata)

and $\chi_{\mathrm{ECR}^{\mathrm{overlap}}}(e) = \mathbb{1}[|S_{\mathrm{hi}}(e)| > 0]$. The held-out pattern overlap rate is

$$\mathrm{ECR}_f^{\mathrm{overlap}} \;=\; \frac{1}{|\mathcal{E}_f|} \sum_{e \in \mathcal{E}_f} \chi_{\mathrm{ECR}^{\mathrm{overlap}}}(e)\,. \tag{2}$$

For the edge–outcome diagnostic, stack normalized edge weights into $\mathbf{w}_f \in \mathbb{R}^{|\mathcal{E}_f|}$ (for ties, use the order exported by the audit log). Let $\mathbf{y}_f \in \mathbb{R}^{|\mathcal{E}_f|}$ carry test-fold outcome summaries aligned to each edge—for example, endpoint averages of correctness on $\mathcal{F}^f_{\mathrm{test}}$ or their logits—computed *after* graph construction (labels are used only for this diagnostic, not for inferring $\mathcal{E}_f$). The primary edge–outcome diagnostic is the Pearson correlation

$$\rho_f \;=\; \mathrm{Corr}(\mathbf{w}_f, \mathbf{y}_f)\,, \tag{3}$$

computed over edges with defined variance; we report the magnitude $|\rho_f|$ in Table 5. Values $|\rho_f| \approx 0$ indicate that retained edge mass is statistically unrelated to test-fold outcome summaries; on the public benchmarks in Table 5, $|\rho|$ ranges from ≈0.09 (XES3G5M) to ≈0.22 (Junyi Academy).

Let $t_{u,i}$ be the timestamp of interaction i for learner u and let τ_u be u's temporal split cutoff for fold f. Write $\mathcal{I}^{>}_{u,f} = \{\, i : t_{u,i} > \tau_u \,\}$ for post-boundary interactions and $\mathcal{V}_{u,f}$ for the subset of those tokens that incorrectly appear in any multiset counted toward $\mathcal{E}_f$ or in graph-derived features at inference for fold f. The within-train Temporal Boundary Mixing Rate is

$$\mathrm{TBMR}_f \;=\; \frac{\sum_u |\mathcal{V}_{u,f}|}{\sum_u |\mathcal{I}^{>}_{u,f}|}\,. \tag{4}$$

Algorithm 1 Edge-provenance leakage audit for fold f

Require: learner sets $\mathcal{U}_f^{\mathrm{tr}}, \mathcal{U}_f^{\mathrm{va}}, \mathcal{U}_f^{\mathrm{te}}$; consumed edges $\mathcal{E}_f$; held-out interactions $\mathcal{H}_f$; edge weights $\mathbf{w}_f$; outcome summaries $\mathbf{y}_f$

Ensure: $\mathrm{ECR}_f^{\mathrm{flag}}$, $\mathrm{ECR}_f^{\mathrm{overlap}}$, $|\rho_f|$, TBMR_f

$\mathrm{ECR}_f^{\mathrm{flag}} \leftarrow \mathbb{1}\big[(\mathcal{U}_f^{\mathrm{tr}} \cap \mathcal{U}_f^{\mathrm{va}}) \cup (\mathcal{U}_f^{\mathrm{tr}} \cap \mathcal{U}_f^{\mathrm{te}}) \cup (\mathcal{U}_f^{\mathrm{va}} \cap \mathcal{U}_f^{\mathrm{te}}) \neq \emptyset\big]$

if $\mathrm{ECR}_f^{\mathrm{flag}} = 1$ **then**

  halt: structural leakage

end if

$\mathrm{ECR}_f^{\mathrm{overlap}} \leftarrow \dfrac{1}{|\mathcal{E}_f|} \sum_{e \in \mathcal{E}_f} \mathbb{1}\big[|S_{\mathrm{hi}}(e)| > 0\big]$ ▷ held-out tokens supporting e

$\rho_f \leftarrow \mathrm{Corr}(\mathbf{w}_f, \mathbf{y}_f)$ ▷ report $|\rho_f|$ in tables

$\mathrm{TBMR}_f \leftarrow \dfrac{\sum_u |\mathcal{V}_{u,f}|}{\sum_u |\mathcal{I}^{>}_{u,f}|}$ ▷ within-train 70% chronological cutoff

return $(\mathrm{ECR}_f^{\mathrm{flag}},\ \mathrm{ECR}_f^{\mathrm{overlap}},\ |\rho_f|,\ \mathrm{TBMR}_f)$

We instantiate τ_u within $\mathcal{F}^f_{\mathrm{train}}$ at the 70% chronological split of each learner's train sequence. TBMR_f is therefore a *within-train temporal stress statistic*: it measures how much edge evidence is supported by the late tail of each learner's *training* sequence after an internal chronological cutoff. It must not be read as cross-fold leakage ($\mathrm{ECR}_f^{\mathrm{flag}}$=0 already certifies learner-disjoint splits). Elevated TBMR_f on dense multi-skill corpora (e.g. XES3G5M) flags reliance on late-train co-assessment patterns, not contamination from validation/test learners.

Algorithm 1 collects the four scalar diagnostics into a single audit pass over a fold. The structural flag $\mathrm{ECR}_f^{\mathrm{flag}}$ is a hard gate that halts the pipeline; the remaining three quantify *throughput* of held-out mass into the consumed edge set ($\mathrm{ECR}_f^{\mathrm{overlap}}$, TBMR_f) and its correlation with outcomes ($|\rho_f|$), none of which reduces to predictive accuracy.

3.3 Graph Representation for Knowledge Tracing

We represent each fold's knowledge structure as a directed graph $G_f = (\mathcal{C}, \mathcal{R}, \mathcal{E}_f)$ where nodes $c \in \mathcal{C}$ are knowledge components (hashed identifiers in the released pipeline), and edges carry relation type $r \in \mathcal{R} = \{\mathrm{pre}, \mathrm{sim}\}$ when that relation is instantiated on the corpus (prerequisite edges on all benchmarks; similarity edges only where the Q-matrix yields non-empty overlap). Prerequisite edges $(i, j) \in E^f_{\mathrm{pre}}$ encode directed precedence inferred from temporal transitions; similarity edges $(i, j) \in E^f_{\mathrm{sim}}$ encode co-assessment structure from the train-fold Q-matrix. Each edge stores provenance fields (fold id, train-only flag, source tag, support count, weight w_{ij}) exported as CSV for audit.

Table 2: Observed graph scale after DAG audit (fold 0) versus theoretical caps. Density is $|E_{\text{pre}}|/(|V_G|(|V_G|-1))$ on prerequisite edges only. $|V_G|$ counts KCs incident to at least one retained edge, not the full vocabulary in Table 4. Even million-scale logs yield sparse, auditable G_f.

Dataset	$\lvert V_G\rvert$	$\lvert E_{\text{pre}}\rvert$	Max out-deg.	Density	Cap regime
Junyi Academy	573	1,139	≤ 5	$\approx$0.0035	K=5000
ASSIST'12	139	261	≤ 5	$\approx$0.0136	K=5000
XES3G5M	455	701	≤ 5	$\approx$0.0034	K=5000

Scalability limits.

Even when train folds contain tens of millions of interactions (Junyi: $\approx$25.9 M rows; Table 4), the exported graph remains bounded: nodes are at most the distinct knowledge component vocabulary $|\mathcal{C}|$; prerequisite edges are capped by a global top-K filter (default K=5,000) and a per-source out-degree cap (default k=5) before DAG pruning (Section 3.4). Transition counting scales linearly in train-fold sequence length; cycle detection operates on the sparse candidate set, not the full $|\mathcal{C}|^2$ adjacency. Table 2 reports observed node and edge counts after pruning on fold 0; density stays well below unity on all public benchmarks. We write $|\mathcal{C}_{\text{vocab}}|$ for the preprocessed KC vocabulary (Table 4) and $|V_G|$ for nodes incident to at least one retained prerequisite edge after DAG audit; $|V_G| \leq |\mathcal{C}_{\text{vocab}}|$.

For graph-augmented KT baselines, G_f is materialised as a row-normalised adjacency matrix $A_f \in \mathbb{R}^{|\mathcal{C}|\times|\mathcal{C}|}$ (`pykt_graph_matrix.py`): prerequisite and similarity relations can be merged or consumed separately depending on the backbone (GKT uses a single KC graph; GIKT additionally uses question–KC bipartite structure from the train fold). Node features are not learned at graph-construction time; the protocol treats G_f as a *structural prior* consumed by downstream KT optimisers. This separation mirrors GNN practice on citation or knowledge graphs [30, 31] but with education-specific relation semantics and fold conditioning.

Technology stack: graph construction is NumPy/pandas/NetworkX; cycle detection uses Johnson's algorithm [32]; KT training optionally delegates to `pyKT` [17] with exported NPZ adjacency. All artefacts are versioned per dataset and fold under `data/graphs/<dataset>/fold_<f>/`.

3.4 Train-only Graph Construction

When $|\mathcal{F}^f_{\text{train}}|$ is very large, the protocol allows an optional *within-train* sampling step to bound edge-inference cost without opening a leakage channel (Figure 1). We draw $\tilde{\mathcal{I}}^f_{\text{tr}} \subseteq \mathcal{F}^f_{\text{train}}$ such that (i) all sampled interactions come from train learners only; (ii) per-knowledge component coverage is preserved by stratifying on KC frequency bins so rare knowledge components are not systematically dropped; and (iii) temporal order is preserved within each sampled learner sequence. All retained edges still carry fold provenance and train-only flags. *All reported experiments in this paper use the full train fold* (sampling disabled); Junyi ($\approx$26 M train interactions) remains tractable under the top-K and per-source top-k filters below.

Prerequisite edges E_{pre}.

For uniformity across heterogeneous benchmarks, we infer E_{pre} from train-only temporal transitions on *all* three datasets, independently of whether an external prerequisite source is also available for that dataset. This deliberate choice decouples the audit pipeline from dataset-specific quality of expert annotations and ensures that every reported edge originates from a single, auditable inference rule. When an expert annotation is available for a benchmark (here, Junyi Academy), the protocol additionally performs ground-truth cross-validation as a separate audit step (Section 4.9).

Concretely, for each train-fold interaction sequence we extract consecutive KC transitions $c_t \to c_{t+1}$ with $c_t \neq c_{t+1}$, count their support $s(c_i, c_j)$, and retain candidate edges in three filtering stages: (i) a support-quantile filter that drops transitions below the q-th percentile (default $q = 0.95$); (ii) a per-source top-k cap (default $k = 5$) that retains, for each source KC, only the k destination knowledge components with highest support; and (iii) a global top-K cap on the number of retained candidate edges (default $K = 5{,}000$). Each retained edge carries the normalized support weight $w_{ij} = s(c_i, c_j)/s_{\max}$ used by the cycle pruning step in Section 3.5. This candidate edge set is then passed unchanged to the DAG audit; cycle pruning operates only on the candidates returned by stages (i)–(iii). Supplementary Figure S5 (Figure 4) diagrams these stages; Figure 3(a) situates the block inside the wider data pipeline (Section B).

Similarity edges E_{sim} and theoretical properties.

We compute E_{sim} from a train-only Q-matrix using either Jaccard or PMI co-occurrence at the item level (default Jaccard with threshold τ=0.10). Let $I(c) \subseteq \mathcal{Q}$ be the train-fold item set tagged with concept c, with $n = |\mathcal{Q}|$ items:

$$\text{sim}_J(c_i, c_j) = \frac{|I(c_i) \cap I(c_j)|}{|I(c_i) \cup I(c_j)|}, \tag{5}$$

$$\text{sim}_{\text{PMI}}(c_i, c_j) = \log \frac{p(c_i, c_j)}{p(c_i)\, p(c_j)},$$

$$p(c) = \tfrac{|I(c)|}{n},$$

$$p(c_i, c_j) = \tfrac{|I(c_i) \cap I(c_j)|}{n}. \tag{6}$$

Jaccard similarity is bounded in $[0, 1]$ and yields no E_{sim} when items are single-skill (ASSISTments 2012; Section 4.8). Full operator definitions and subgraph sampling details are in Supplementary Section S4. All graph artefacts are materialised per dataset and fold.

3.5 DAG Audit

For each fold-specific prerequisite edge set, we check whether E_{pre} is acyclic. If cycles are detected, we iteratively remove the lowest-confidence edge in a detected cycle and re-run the check until a topological ordering exists. Cycle detection uses a bounded simple-cycle enumeration procedure based on Johnson's algorithm for enumerating elementary circuits in directed graphs [32], capped at 100 representative cycles per

Algorithm 2 Train-only prerequisite graph construction with DAG audit

Require: train fold $\mathcal{F}^f_{\text{train}}$; quantile q; per-source cap k; global cap K
Ensure: acyclic edge set E^f_{pre} with provenance

$s \leftarrow \emptyset$ ▷ support counter over KC transitions
for each learner sequence $(c_1, \ldots, c_T) \in \mathcal{F}^f_{\text{train}}$ **do**
  for $t \leftarrow 1$ to $T-1$ **with** $c_t \neq c_{t+1}$ **do**
    $s[(c_t, c_{t+1})] \leftarrow s[(c_t, c_{t+1})] + 1$
  end for
end for
$\theta \leftarrow q$-th percentile of $\{s[\cdot]\}$ ▷ (i) support-quantile filter
$C \leftarrow \{(i,j) : s[(i,j)] \geq \theta\}$
$C \leftarrow \bigcup_i \textsc{TopK}_j(C_i, k)$ by support ▷ (ii) per-source top-k
$C \leftarrow \textsc{TopK}(C, K)$ by support ▷ (iii) global top-K
for $(i,j) \in C$ **do**
  $w_{ij} \leftarrow s[(i,j)] / \max_{(a,b)\in C} s[(a,b)]$; tag $(f, \textit{train-only})$
end for
$E^f_{\text{pre}} \leftarrow C$
while E^f_{pre} contains a cycle **do** ▷ DAG audit
  $Z \leftarrow \textsc{JohnsonCycles}(E^f_{\text{pre}},\ \text{cap} = 100)$
  $e^\star \leftarrow \arg\min_{e \in \bigcup Z} w_e$; remove $e^\star$; log $e^\star$
end while
return E^f_{pre} with topological order

detection round. We use this cap because the number of elementary cycles can grow rapidly with graph density; exhaustive enumeration is unnecessary for our audit, since the pruning loop only requires representative cycle witnesses. The detector cap does not affect correctness: the pruning loop terminates only when no further cycle can be detected, so the final retained graph is acyclic. Each removed edge is written to `<dataset>\_dag\_pruning\_log.csv` (repository reports/) with its source, target, confidence score, and pruning reason. The audit reports node count, raw edge count, pruned edge count, retained edge count, root count, leaf count, and final topological-sort status. The claim is not that the retained DAG is the unique correct prerequisite hierarchy, but that the pruning rule and edge provenance are explicit and reproducible.

Algorithm 2 states the construction and audit steps together. Counting and filtering touch only $\mathcal{F}^f_{\text{train}}$, so no held-out interaction can support a retained edge; the cycle-pruning loop terminates only when no further cycle is detected, guaranteeing an acyclic E^f_{pre}.

3.6 DAG Disruption Rate and Augmentation Operators

To quantify the structural effect of graph augmentations on prerequisite DAGs, we define the DAG Disruption Rate (DDR). Let A be an augmentation operator and $A(E_{\text{pre}}, p)$ the edge set obtained at perturbation strength p. Let $\text{pre}(A(E_{\text{pre}}, p)) \subseteq E_{\text{pre}}$ be the original prerequisite edges that remain with their original direction, and let

Algorithm 3 DAG Disruption Rate (DDR)

Require: prerequisite edges E_{pre}; augmentation operator A; strength p
Ensure: $\text{DDR}(A, p) \in [0, 1]$
$E' \leftarrow A(E_{\text{pre}}, p)$ ▷ apply operator at strength p
pre $\leftarrow \{(i, j) \in E_{\text{pre}} : (i, j) \in E'\}$ ▷ kept, same direction
rev $\leftarrow \{(i, j) \in E_{\text{pre}} : (j, i) \in E' \textbf{ and } (i, j) \notin E'\}$ ▷ reversed
return $\big(|E_{\text{pre}} \setminus \text{pre}| + |\text{rev}|\big) \,/\, |E_{\text{pre}}|$

$\text{rev}(A(E_{\text{pre}}, p))$ be the edges that appear with the opposite direction. We define

$$\text{DDR}(A, p) = \frac{|E_{\text{pre}} \setminus \text{pre}(A(E_{\text{pre}}, p))| + |\text{rev}(A(E_{\text{pre}}, p))|}{|E_{\text{pre}}|}. \tag{7}$$

We evaluate four label-agnostic families from GraphCL [19] (`attr_mask`, `edge_drop`, random-walk `subgraph`, `node_drop`) plus one prerequisite-preserving operator (`prereq_preserve`) that removes only transitively redundant edges at the same per-edge budget as `edge_drop` (Section 4.6; formal definitions in Supplementary Section S4). Each operator is swept over $p \in \{0.05, 0.10, 0.20, 0.30\}$ with three random seeds on each of three train folds ($3 \times 3 = 9$ runs per setting); reported means aggregate across those runs. DDR is a structural diagnostic only: it measures how much prerequisite structure is removed or reversed by an augmentation, not whether a downstream KT model becomes better or worse.

Algorithm 3 states the computation. An edge counts as disrupted if it is removed outright or re-appears with reversed direction, so DDR captures both deletion and direction flips of prerequisite precedence.

3.7 Cold-start KC Stratification

We define cold-start at the KC level, distinct from user/item cold-start in recommender systems [16]. For fold f, let $n_f(c) = |\{(u, t, q, c, y) \in \mathcal{F}^f_{\text{train}}\}|$ be the number of train-fold interactions tagged with knowledge component c. Each KC is assigned to one of four frequency strata:

$$\begin{aligned}
\mathcal{S}_1 &: \; n_f(c) < 20, \\
\mathcal{S}_2 &: \; 20 \le n_f(c) < 100, \\
\mathcal{S}_3 &: \; 100 \le n_f(c) < 500, \\
\mathcal{S}_4 &: \; n_f(c) \ge 500.
\end{aligned}$$

Strata are assigned from train-fold counts only; test rows inherit their knowledge component stratum and aggregate AUC/ACC/NLL per stratum. KCs with zero train-fold interactions are absent from the frequency table and therefore excluded from stratum aggregates unless they appear under *out of range* when a held-out knowledge component cannot be mapped to the four canonical bins. We use *cold-start* throughout in this *frequency-stratum* sense (few train interactions), distinct from zero-interaction

Table 3: Pros and cons of common KC-graph construction practices versus the leakage-controlled audit protocol (Section 3).

Practice	Advantages	Disadvantages	Leakage control
Full-log pooled graph (GKT/SKT default)	Simple; maximises transition counts	Held-out learners can support edges; ranking may inflate graph backbones	No
Train-only inference, no DAG audit	Fast; fold-clean edge evidence	Cycles may persist silently; augmentations unmeasured	Partial
Expert-only DAG (e.g. Junyi annotation)	Pedagogically curated structure	Unavailable on most benchmarks; not behavioural	N/A
End-to-end learned graph	Adaptive to data	Opaque provenance; hard to audit against splits	Unclear
Audit protocol (this paper)	Fold-specific provenance; bounded sparse G_f; DDR + cold-start strata	Overhead; behavioural–expert edge F1 can be low	**Yes**

or zero-shot cold-start splits in the broader dissertation roadmap. This exposes tail-knowledge component behaviour that aggregate metrics hide and links rare-knowledge component edge support to $\mathrm{ECR}_f^{\mathrm{overlap}}$ (Table 1).

3.8 Ground-Truth Cross-Validation

When a benchmark provides an expert prerequisite annotation, the protocol performs an additional, optional audit step: comparing the train-only inferred E_{pre} against the expert edge set $E_{\mathrm{pre}}^{\mathrm{expert}}$. The procedure has four stages: (i) load the expert annotation and filter rows by a prerequisite-score threshold θ to obtain a directed expert edge set; (ii) align expert KC identifiers to the pipeline's hashed `kc_id` space via a deterministic name-to-id mapping reconstructed from the preprocessing hash function; (iii) sweep top-K cutoffs over the inferred edge set and compute, at each K: edge precision/recall/F1, direction agreement, reachability F1 (over transitive closure), and node Jaccard; (iv) extract qualitative examples of disagreement in three categories: edges present only in inferred, edges present only in expert, and edges with opposite direction. The procedure is read-only with respect to the pipeline; no inferred edge is added or removed based on the comparison.

3.9 Comparison with common KC-graph practices

Table 3 contrasts how practitioners typically obtain KC graphs against the audit protocol developed here. The comparison is methodological, not a predictive leaderboard: our claim is that graph *provenance* should be a first-class experimental factor.

Adopt this protocol when (i) graph-augmented KT results inform deployment or publication comparisons, (ii) the concept graph is inferred from the same log used for evaluation, or (iii) reviewers or regulators require explicit leakage and cold-start reporting. Expert-only or full-log graphs may suffice for exploratory prototyping, but

Table 4: Dataset statistics used in the P0 diagnostic protocol. Junyi Academy counts reflect the Chang et al. problem-level log (`junyi_ProblemLog_original.csv`) after preprocessing; #items and #KCs coincide because both map to the exercise column. Synthetic C2/C5 are companion sanity logs shipped with the preprocessing scripts.

Dataset	# learners	# items	# KCs	# interactions	Avg. seq. len.	Has DAG
Junyi Academy	247,606	722	722	25,925,992	104.7	Yes
ASSISTments 2012	29,018	53,086	265	2,711,602	93.4	No
XES3G5M	18,066	7,652	865	6,413,353	355.0	Yes
Synthetic C2	4,000	50	2	200,000	50.0	No
Synthetic C5	4,000	50	5	200,000	50.0	No

they should not be treated as audited benchmarks without provenance logs. When both expert and behavioural signals exist, ground-truth cross-validation (C4) supports treating them as complementary relations rather than substitutes (Section 4.9).

4 Experiments

4.1 Datasets

Benchmark roles.

XES3G5M [24] is our *primary* public benchmark for model-comparison claims: low consecutive KC-repeat (≈21%, Supplementary Section S3), non-saturated headline AUC (≈0.875 for *simpleKT*), and the widest separation among graph backbones under train-only graphs. **ASSISTments 2012** is *secondary*: it stress-tests leakage ablation, cold-start strata, and DAG audit on a single-skill Q-matrix ($|E_{\text{sim}}|$=0). **Junyi Academy** [22] is a *saturated sanity* corpus for ground-truth cross-validation and dense E_{pre} topology; near-ceiling AUC (≈0.98–0.984) makes it unsuitable for headline graph-benefit claims. Synthetic C2/C5 are controlled tiny-graph sanity checks.

We instantiate the protocol on all five corpora. Junyi ships an expert-curated prerequisite annotation at the exercise-pair level. *In the main inference branch we do not consume this expert DAG*: E_{pre} for Junyi is built by the same train-only temporal inference rule used for ASSISTments and XES3G5M (Section 3.4). The expert DAG enters only the read-only ground-truth cross-validation sub-protocol (Section 4.9). ASSISTments provides a Q-matrix but no external prerequisite graph. XES3G5M provides hierarchical KC metadata parsed at the sequence level; E_{pre} is inferred by the shared train-only rule. Table 4 summarises the preprocessed datasets. The public benchmarks differ substantially in interaction volume, KC vocabulary size, and item-to-KC granularity. This heterogeneity is useful for auditing because the protocol must remain leakage-controlled under both curated and inferred graph settings. We use learner-based temporal splits with ratios $(0.7, 0.1, 0.2)$ and base split seed 42; the three reported folds use split seeds 42, 43, and 44 (fold f uses seed $42+f$). The anchored GKT DDR multi-seed sweep instead uses base seeds $\{42, 17, 1234\}$ (nine folds). All edge provenance is regenerated for each run.

Table 5: Direct leakage diagnostics per dataset (mean ± std over three folds). $\mathrm{ECR}_{\mathrm{flag}}$: learner-overlap indicator (Eq. 1). $\mathrm{ECR}_{\mathrm{overlap}}$: held-out pattern overlap (Eq. 2). $|\rho|$: edge–outcome Pearson correlation magnitude (Eq. 3). TBMR: within-train temporal mixing (Eq. 4), not a train/test violation. Edge share > 50%: fraction of retained edges whose held-out transition share exceeds 0.5; share p90: 90th percentile of per-edge held-out shares (Supplementary artefact `edge_share_summary.csv`).

Dataset	$\mathrm{ECR}_{\mathrm{flag}}$	$\mathrm{ECR}_{\mathrm{overlap}}$	$\lvert\rho\rvert$	TBMR	> 50%	p90
ASSISTments 2012	0.000 ± 0.000	1.000 ± 0.000	0.109 ± 0.006	0.201 ± 0.001	0.000 ± 0.000	0.334 ± 0.009
Junyi Academy	0.000 ± 0.000	1.000 ± 0.000	0.224 ± 0.001	0.176 ± 0.001	0.005 ± 0.001	0.355 ± 0.009
Synthetic C2	0.000 ± 0.000	1.000 ± 0.000	0.000 ± 0.000	0.733 ± 0.000	—	—
Synthetic C5	0.000 ± 0.000	1.000 ± 0.000	0.000 ± 0.000	0.133 ± 0.000	—	—
XES3G5M	0.000 ± 0.000	0.988 ± 0.001	0.088 ± 0.010	0.756 ± 0.001	0.079 ± 0.001	0.339 ± 0.001

Table 6: DAG audit summary (fold 0). *Raw* and *Final* are the prerequisite edge counts before and after lowest-confidence cycle pruning inside `dag_audit`; *Pruned* is the difference. The *Cycles* column reports representative cycles found before pruning (cap 100). All corpora pass topological sort with zero cycles after pruning on fold 0.

Dataset	$\lvert V\rvert$	Raw $\lvert E_{\mathrm{pre}}\rvert$	Pruned	Final $\lvert E_{\mathrm{pre}}\rvert$	Roots	Leaves	Cycles (cap)
Junyi Academy	573	2,152	1,013	1,139	90	92	≥ 100
ASSIST'12	139	413	152	261	34	23	≥ 100
XES3G5M	455	1,162	461	701	129	73	≥ 100
Synthetic C2	2	2	1	1	1	1	1
Synthetic C5	3	2	0	2	1	1	0

Benchmark repetitiveness.

Learner-disjoint splits remove cross-fold leakage but not within-log repetition shortcuts. Junyi and ASSISTments exhibit high consecutive KC-repeat rates (0.66–0.76), whereas XES3G5M repeats only 21% of the time (Supplementary Section S3). On the repetition-heavy corpora, *simpleKT* reaches AUC ≈0.968–0.984 despite weak edge–outcome correlation (Table 5), whereas XES3G5M remains near 0.875 under the same protocol. This pattern is associated with higher absolute deep-KT AUC where KC repetition is high, is consistent with the near-null train-only versus full-log ablation (Section 4.5), and is why we anchor model-comparison claims on XES3G5M rather than on Junyi/ASSIST headline AUC.

4.2 Leakage and DAG Audit

Table 5 summarises the direct leakage metrics on the released runs (mean ± std over three folds). $\mathrm{ECR}^{\mathrm{flag}}$ is identically zero on every dataset (fold means), confirming learner-disjoint splits and structural leakage freedom at the evaluation protocol level; $\mathrm{ECR}^{\mathrm{overlap}}$ stays near unity because recurrent transition patterns reappear among held-out learners even though edges are inferred strictly from train folds. Per-edge

held-out shares remain mostly modest (p90 typically $\ll 1$ on XES3G5M despite overlap ≈ 0.99; Table 5), so the saturated overlap column should be read together with the share-distribution columns rather than as proof that every edge is dominated by held-out mass. The edge–outcome correlation magnitude is weak on all three public corpora ($|\rho| \approx 0.09$–0.22; Table 5), so retained edge mass is only weakly aligned with test-fold outcome summaries; ρ is undefined on degenerate synthetic graphs where edge-weight variance vanishes. TBMR is reported as the within-train temporal stress statistic of Section 3.2 (the share of edge evidence drawn from the late, post-τ_u tail of each learner's train sequence); it ranges from ≈ 0.18–0.20 on Junyi and ASSISTments to ≈ 0.76 on XES3G5M (Table 5). Because folds are learner-disjoint ($\mathrm{ECR}^{\mathrm{flag}}=0$), these values quantify reliance on late train transitions rather than any cross-split contamination. We read $\mathrm{ECR}^{\mathrm{flag}}=0$ and provenance logs as **primary** pass/fail checks; overlap, ρ, and TBMR as **sanity/stress** indicators that can be near-degenerate under honest splits (Section 5.3). The leakage audit reports that all retained graph edges are train-only under the current fold configuration. Table 6 reports the DAG audit on fold 0 for all five corpora. The protocol produces non-empty acyclic prerequisite graphs after lowest-confidence cycle pruning on every dataset except the degenerate Synthetic C2 case ($|E_{\mathrm{pre}}|=1$ after pruning).

On Junyi fold 0, 2,152 candidate edges prune to 1,139 acyclic edges; ASSISTments and XES3G5M retain 261 and 701 edges respectively after pruning (Table 6). Even train-only inference can induce cycles; the audit makes the pruning rule explicit rather than silently fixing the graph.

Supplementary Figure S1 (Figure 9) illustrates the *shape* of retained prerequisite structure on all three public benchmarks, including a high-support Junyi chain after DAG audit. These plots are not used for inference; they help reviewers compare graph density with DDR and baseline behaviour in later sections.

4.3 Controlled test-fold edge injection

This experiment is the controlled realisation of the high-throughput cell of the two-factor model (Table 7): we raise contamination *throughput* on purpose and observe which backbones move. We deliberately contaminate the XES3G5M fold-0 train-only prerequisite builder with 5% and 20% of test-fold interactions (Supplementary Table S17). Candidate-edge count ($|E_{\mathrm{pre}}|$: 1162→1378→1417; counted *before* DAG pruning, whereas Table 6 reports the 701 edges retained *after* pruning on the clean fold) and TBMR_f (0.754→0.776→0.779) increase monotonically with the injection rate, while $\mathrm{ECR}_f^{\mathrm{flag}}$ remains 0 because learner-id sets stay disjoint: flag-level checks alone cannot detect this contamination channel, which motivates the tiered audit. $\mathrm{ECR}_f^{\mathrm{overlap}}$ is already saturated at 1.0 on this transition-dense corpus (Section 3.1), so it carries no additional signal here. Retraining on the injected graphs uses the *reduced* graph-backbone budgets in Table S15 (10 epochs, smaller batches; Supplementary Table S18), not the 30-epoch sequence cap—so absolute AUC levels are not directly comparable to headline Table 8 rows. The qualitative pattern we emphasise is *decoupling*: builder-side indicators move while fold-0 test AUC stays near flat for all three backbones ($|\Delta\mathrm{AUC}| \leq 0.010$ at 20% injection). *simpleKT* moves by +0.0004 (0.8744→0.8748); GIKT by +0.0002 (0.8776→0.8778); GKT *dips* slightly by

Table 7: Two-factor interaction: graph-mediated leakage shifts AUC only in the high-throughput × high-reliance cell. Rows are contamination *throughput* into the consumed edge set; columns are backbone *reliance* on the graph. Entries are observed ΔAUC on XES3G5M. Three of four cells are ≈0; harm concentrates in one cell, and the realistic operating point (full-log pooling under our filter) sits in the low-throughput row.

	low reliance (*simpleKT*)	high reliance (GKT, GIKT)
low throughput (train-only vs. full-log pooling)	≈0	≤0.003 (Table 10)
high throughput (20% test-fold injection)	≈0 (0.8744→0.8748)	≈0 (GKT −0.010; GIKT +0.0002; Table S18)

Table 8: Diagnostic baseline AUC summary (train-only graphs; three-fold learner CV means ± std). XES3G5M is the primary model-comparison column; Junyi is a saturated sanity corpus. Per-dataset ACC/NLL and 95% bootstrap confidence intervals are in Supplementary Tables S1–S5; fold-level detail in Table 9.

Model	XES 3G5M	ASSIST 2012	Junyi	Synth C2	Synth C5
BKT	0.636 ± 0.002	0.669 ± 0.002	0.780 ± 0.001	0.835 ± 0.003	0.791 ± 0.002
AKT	0.875 ± 0.001	0.968 ± 0.001	0.984 ± 0.004	0.873 ± 0.003	0.822 ± 0.002
DKT	0.858 ± 0.000	0.963 ± 0.001	0.984 ± 0.004	0.862 ± 0.006	0.826 ± 0.001
simpleKT	0.875 ± 0.000	0.968 ± 0.000	0.984 ± 0.004	0.875 ± 0.004	0.829 ± 0.002
GKT	0.834 ± 0.001	0.961 ± 0.001	—	0.873 ± 0.005	0.810 ± 0.004
GIKT	0.878 ± 0.000	0.967 ± 0.000	0.982 ± 0.000	0.875 ± 0.005	0.820 ± 0.004
SKT	0.625 ± 0.001	0.953 ± 0.001	0.979 ± 0.006	0.708 ± 0.126	0.810 ± 0.003
DyGKT	0.752 ± 0.001	0.952 ± 0.001	0.978 ± 0.000	0.710 ± 0.136	0.826 ± 0.002
DGEKT	0.841 ± 0.000	0.960 ± 0.001	0.981 ± 0.000	0.710 ± 0.130	0.830 ± 0.002

−0.010 (0.8346→0.8243). Metric-level contamination is detectable in builder mass and TBMR_f before practitioners would infer leakage from AUC alone.

Table 7 assembles the interaction from evidence already in the paper: GKT shows ≈0 under both full-log pooling (Table 10) and deliberate injection (Table S18) while builder-side throughput indicators rise (Table S17), so harm behaves multiplicatively (throughput × reliance) and AUC is a late alarm. Section 4.7 corroborates the reliance axis independently: near-total graph destruction moves GKT by 0.07–0.09 AUC on XES3G5M but ≤0.003 on ASSISTments and for DGEKT (Table 7).

4.4 Baseline Diagnostic

Table 8 sanity-checks preprocessing, learner splits, train-only graph export, and pyKT wiring across five corpora; it is not a SOTA claim. We read the table through the benchmark roles of Section 4.1. On the *primary* benchmark XES3G5M we report three **primary comparison backbones**: sequence-only *simpleKT*/AKT (≈0.875), stock pyKT GKT (≈0.834), and native GIKT (≈0.878). GKT trails *simpleKT* by ≈0.041 mean fold AUC on XES3G5M (95% CI $[-0.044, -0.038]$; Table S16). Extended-training configuration sensitivity for GKT is consolidated in Section 5.3

(Table S21). GIKT slightly exceeds the sequence leader (0.878 vs. 0.875; paired interval $[+0.003, +0.003]$, identical at three decimals on all three folds in Table 9—we report magnitudes rather than over-interpreting the degenerate interval width). SKT and DyGKT also appear in Tables 8 and 9 for fork wiring sanity only; they are excluded from primary ranking claims (Section 5.1). On ASSISTments (secondary), GKT trails by ≈0.007 AUC with near-ceiling scores for all leaders. On saturated Junyi, all deep models cluster near 0.98–0.984 and are poor discriminators of graph benefit. Per-dataset ACC/NLL and bootstrap intervals are in Supplementary Tables S1–S5.

Training-configuration disclosure and comparison scope.

Training configurations and comparison scope are reported in Supplementary Table S15. Sequence-only checkpoints (AKT, DKT, *simpleKT*) use stock `pyKT` configurations (`lr`=10^{-3}, `max_seq_len`=200, batch 64, maximum 30 epochs). GKT is evaluated through the `pyKT` graph branch (batch 4, maximum 10 epochs under the primary attested budget; consumes exported $E_{\text{pre}}/E_{\text{sim}}$). Native GIKT (`src/models/gikt.py`) uses `emb_size`=64, `hidden_dim`=128, the same `lr`, batch 16, and maximum 10 epochs. We do not claim bit-identical engineering parity across codebases; we align processed splits, data loaders, evaluation hooks, and early stopping (patience 5, best validation AUC). A one-step label-index misalignment in an early GIKT head was caught by fold-wise AUC inconsistency and corrected before Table 9 (Section 5.3). No family receives a hyperparameter search. Maximum epoch caps differ in the primary release: sequence checkpoints train up to a maximum of 30 epochs (batch 64) while graph backbones use maximum 10-epoch budgets with smaller batches (Table S15), reflecting higher per-epoch cost; see Section 5.3 and Table S21 for the exploratory extended-training check. The residual deficit remains an *observational* benchmarking boundary on XES3G5M under this protocol, not a causal claim that graph structure is uninformative. Relative to the literature default of pooling transition statistics before learner splitting in GKT/SKT pipelines [8, 9], our train-only graphs isolate a provenance factor those papers do not audit.

Table 9 reports three-fold learner-CV means ± std and Wilcoxon p-values vs. *simpleKT*. Table 8 provides the same mean ± std format across datasets. With only three paired folds the Wilcoxon test is underpowered (minimum two-sided p=0.25); paired t-test p-values on the same fold AUCs appear in Supplementary Table S11. We treat fold-level ΔAUC magnitudes (Table 9) and Table S16 intervals as the primary inferential evidence for the XES3G5M primary trio on this protocol; exploratory ANOVA summaries appear only in Supplementary Tables S19–S20. The resulting *model-specific* ordering on XES3G5M within the primary trio—pyKT GKT as the under-performer relative to *simpleKT* across all three folds (split seeds 42–44), native GIKT competitive or slightly above—is developed in Section 5.1.

4.5 Graph Construction Ablation (Train-only vs. Full Log)

We contrast fold-specific train-only graph exports (`e\_pre\_train\_only.csv`, `e\_sim\_train\_only.csv`) against a *single* graph inferred from the entire preprocessed log (`data/processed/<dataset>/full\_log/`), holding learner splits and evaluation

Table 9: Three-fold learner CV baseline summary (train-only graphs, valid+test positions). Mean ± std over folds; p_W: Wilcoxon signed-rank test vs. *simpleKT* (paired fold AUCs).

Dataset	Model	AUC	ACC	NLL	p_W
ASSIST '12	*simpleKT*	0.968 ± 0.000	0.916 ± 0.001	0.173 ± 0.001	—
	GKT	0.961 ± 0.001	0.907 ± 0.001	0.185 ± 0.001	0.250
	GIKT	0.967 ± 0.000	0.915 ± 0.001	0.175 ± 0.001	0.250
	SKT	0.953 ± 0.001	0.898 ± 0.001	0.198 ± 0.001	0.250
	DyGKT	0.952 ± 0.001	0.899 ± 0.001	0.200 ± 0.002	0.250
	DGEKT	0.960 ± 0.000	0.907 ± 0.001	0.186 ± 0.001	0.250
XES3G5M	*simpleKT*	0.875 ± 0.000	0.858 ± 0.001	0.324 ± 0.001	—
	GKT	0.834 ± 0.001	0.842 ± 0.000	0.364 ± 0.001	0.250
	GIKT	0.878 ± 0.000	0.860 ± 0.000	0.321 ± 0.001	0.250
	SKT	0.625 ± 0.001	0.800 ± 0.001	0.482 ± 0.001	0.250
	DyGKT	0.752 ± 0.001	0.813 ± 0.001	0.431 ± 0.001	0.250
	DGEKT	0.841 ± 0.000	0.844 ± 0.000	0.358 ± 0.001	0.250
Junyi Academy	*simpleKT*	0.984 ± 0.003	0.951 ± 0.008	0.121 ± 0.019	—
	GIKT	0.982 ± 0.000	0.946 ± 0.000	0.133 ± 0.001	1.000
	SKT	0.979 ± 0.005	0.938 ± 0.011	0.139 ± 0.023	0.250
	DyGKT	0.978 ± 0.000	0.934 ± 0.000	0.151 ± 0.001	0.250
	DGEKT	0.981 ± 0.000	0.943 ± 0.000	0.137 ± 0.001	0.250

Table 10: Train-only versus full-log graph ablation summary (fold 0). Columns report the largest absolute ΔAUC and ΔACC (full-log minus train-only) across GKT, GIKT, SKT, DyGKT, and DGEKT. Per-model train-only/full-log pairs are in Supplementary Tables S6–S10.

Dataset	max |ΔAUC—	max |ΔACC—	Largest shift (model)
Junyi Academy	0.003	0.008	SKT (ΔAUC=−0.003)
ASSISTments 2012	0.000	0.000	all models ≈ 0
XES3G5M	0.003	0.001	DyGKT (ΔAUC=−0.003)
Synthetic C2	0.001	0.001	GKT/DGEKT
Synthetic C5	0.002	0.002	GKT (ΔAUC=+0.002); GIKT (ΔACC=−0.002)

code fixed. Only the static graph artefact fed into the pyKT graph branch differs; KT optimisation still uses train-fold sequences only.

Table 10 summarises the largest train-only versus full-log shifts across GKT, GIKT, SKT, DyGKT, and DGEKT (fold 0). On the three public benchmarks, $|\Delta\text{AUC}| \leq 0.003$ and $|\Delta\text{ACC}| \leq 0.008$ (Junyi SKT reaches the ACC bound); Synthetic C5 shows the largest released shift ($|\Delta\text{AUC}| = 0.002$ for GKT). Per-model pairs appear in Supplementary Tables S6–S10. Under our support quantile (q=0.95), top-k cap, and K=5000 filter, pooling learners before edge inference rarely changes surviving transitions on public benchmarks. We read this null shift as a *measurement of operating regime*, not as evidence that leakage is harmless: because most edges that survive full-log pooling already survive train-only construction, held-out mass has low *throughput* into the consumed edge set. The ≤0.003 shift therefore places these three benchmarks in the low-throughput row of Table 7; it is the aggressive filter that throttles the channel

Table 11: Mean ± std DDR across three seeds and three train folds for the five augmentation families and four perturbation strengths (9 runs per cell). Subgraph sampling is random-walk based (Section 3.6); `prereq_preserve` removes only transitively redundant edges at the same per-edge budget as `edge_drop`. Lower is less disruptive.

Dataset	Family	p=0.05	p=0.10	p=0.20	p=0.30
Junyi Academy	attr_mask	0.000 ± 0.000	0.000 ± 0.000	0.000 ± 0.000	0.000 ± 0.000
	subgraph	0.013 ± 0.000	0.063 ± 0.009	0.161 ± 0.017	0.266 ± 0.021
	prereq_preserve	0.047 ± 0.002	0.088 ± 0.002	0.186 ± 0.004	0.278 ± 0.007
	edge_drop	0.050 ± 0.001	0.094 ± 0.002	0.200 ± 0.005	0.301 ± 0.008
	node_drop	0.107 ± 0.019	0.196 ± 0.026	0.384 ± 0.014	0.523 ± 0.010
ASSIST'12	attr_mask	0.000 ± 0.000	0.000 ± 0.000	0.000 ± 0.000	0.000 ± 0.000
	subgraph	0.012 ± 0.002	0.028 ± 0.003	0.071 ± 0.007	0.188 ± 0.023
	prereq_preserve	0.048 ± 0.010	0.087 ± 0.011	0.179 ± 0.014	0.262 ± 0.017
	edge_drop	0.055 ± 0.011	0.100 ± 0.013	0.212 ± 0.011	0.309 ± 0.009
	node_drop	0.092 ± 0.029	0.193 ± 0.027	0.389 ± 0.090	0.530 ± 0.078
XES3G5M	attr_mask	0.000 ± 0.000	0.000 ± 0.000	0.000 ± 0.000	0.000 ± 0.000
	subgraph	0.022 ± 0.002	0.058 ± 0.005	0.145 ± 0.011	0.240 ± 0.010
	prereq_preserve	0.038 ± 0.005	0.072 ± 0.006	0.158 ± 0.011	0.233 ± 0.006
	edge_drop	0.049 ± 0.005	0.095 ± 0.005	0.204 ± 0.009	0.299 ± 0.002
	node_drop	0.110 ± 0.022	0.196 ± 0.027	0.386 ± 0.035	0.535 ± 0.027

here, not a general property of leakage. Deliberate injection raises builder throughput (Table S17) without a matching fold-0 AUC shift under Table S15 budgets (Table S18; Section 4.3). Train-only construction is thus *not* penalising headline metrics relative to this deliberate full-log variant on those corpora; reporting which regime a corpus occupies, with evidence, is itself a useful audit output, and absence of a headline shift here does not prove absence of leakage in general.

4.6 DAG Disruption Rate

Supplementary Figure S2 (Figure 10) plots DDR versus perturbation strength p on all three benchmarks; the main text retains the tabular summary only.

Figure 10 and Table 11 show five consistent regimes across the three benchmarks. First, attribute masking leaves E_{pre} structurally unchanged and therefore gives DDR = 0 across perturbation strengths by construction. Second, random-walk subgraph sampling preserves connectivity inside the sampled subgraph and disrupts E_{pre} *sub-linearly* in p: because edges are removed only at the boundary of the visited subgraph, the disruption is consistently smaller than under edge dropping at the same p (e.g. on Junyi, DDR = 0.16 for subgraph vs. 0.20 for edge drop at $p = 0.20$). Third, edge dropping disrupts prerequisite structure approximately linearly in p, as expected when each edge is removed independently. Fourth, node dropping produces the largest disruption because removing a node also removes all incident prerequisite edges; at $p = 0.30$, node dropping disrupts roughly half of the prerequisite structure on all three benchmarks (DDR $\approx$ 0.52–0.54).

DDR *is an operator-level structural diagnostic, not a tautology.*

A natural objection to the four label-agnostic families is that their DDR ordering merely re-states their definitions: an operator that deletes more edges disrupts more structure. To test whether DDR *discriminates* structure-aware from structure-agnostic augmentation rather than re-describing edge counts, we add a fifth, prerequisite-preserving operator (`prereq_preserve`). It draws the same per-edge budget p as `edge_drop`, but only removes edges that are *transitively redundant*—those whose endpoints remain connected through an alternative directed path—so that every reachability/precedence relation and the entire transitive-reduction backbone are preserved. Crucially, unlike `attr_mask`, this operator is *not* DDR=0 by construction: its disruption is an empirical quantity equal to p times the redundant-edge fraction of the graph. At every dataset and strength, `prereq_preserve` is strictly less disruptive than `edge_drop` at a matched budget (at $p = 0.30$, 0.278 vs. 0.301 on Junyi, 0.262 vs. 0.309 on ASSISTments, and 0.233 vs. 0.299 on XES3G5M). The size of this gap is *data-dependent*—a $\sim 8\%$ relative reduction on Junyi but $\sim 22\%$ on XES3G5M—and is determined by graph redundancy rather than by the operator definition. Because temporal-precedence inference generates many transitive shortcuts (if a precedes b precedes c, the edge $a \to c$ is also counted), the inferred graphs are highly redundant ($\approx$ 78–92% of edges lie off the transitive reduction), which both explains the modest gap and shows that DDR responds to operator structure-awareness in a quantitative, non-tautological way. DDR can therefore serve as a concrete design target: an augmentation that respects prerequisite directionality is measurably rewarded with lower disruption at the same perturbation budget. On a graph-reliant backbone this structural reward also carries downstream—`prereq_preserve` costs the least test AUC at a matched budget on GKT/XES3G5M—so the design target is validated in accuracy, not only in structure, whenever the backbone actually consumes the graph (Section 4.7).

The relative ordering DDR(attr_mask) $<$ DDR(subgraph) $\lesssim$ DDR(prereq_preserve) $<$ DDR(edge_drop) $<$ DDR(node_drop) is consistent across all three benchmarks and all four perturbation strengths. We read these results as a structural diagnostic, not as a claim that any augmentation is beneficial or harmful for a downstream KT learner. They indicate that prerequisite-aware augmentation design is needed when graph augmentations are applied to educational DAGs: node-level operators are an order of magnitude more disruptive than attribute-level operators, with subgraph sampling and the prerequisite-preserving operator occupying an intermediate, structure-preserving regime.

4.7 DDR Downstream Stress Test

We retrain graph-consuming backbones on perturbed graphs and ask whether structural disruption (DDR) predicts accuracy loss. Because a null DDR→AUC relationship is ambiguous—it can mean either "DDR does not predict AUC" or "this backbone ignores the graph"—we gate every interpretation on a *manipulation check*: an anchor at p=0.90 that destroys almost the entire graph (DDR$\approx$0.9–1.0). A backbone whose AUC does not move even under total destruction is graph-inert, and its DDR→AUC correlation carries no downstream meaning.

Low-reliance anchors (DGEKT everywhere; GKT on ASSISTments).

Table 12 reports the original DGEKT sweep on ASSISTments 2012 and XES3G5M. Despite large structural disruption (`node_drop` mean DDR 0.493–0.555 at p=0.30), DGEKT AUC changes stay near zero and pooled DDR–AUC-drop correlations are weak (r=0.13–0.15). GKT on ASSISTments behaves the same way (Table 13, second block): destroying the graph entirely at the manipulation-check anchor (`node_drop` p=0.90, DDR=0.995) moves AUC by only 0.0021 (0.9630→0.9610). These cells *fail* the manipulation check; we read them as *low-reliance anchors* (reliance ≈0), not as evidence about DDR. This is exactly the low-reliance column of the two-factor account (Table 7): where the backbone does not consume the graph, no structural metric can shift accuracy, and DDR's value is confined to structure and reproducibility.

Graph-reliant backbone (GKT on XES3G5M):* DDR *predicts AUC loss.

The same protocol on GKT/XES3G5M (Table 13, first block; seeds {42, 17, 1234}, nine folds) tells the opposite story. GKT *passes* the manipulation check decisively: near-total destruction drops AUC from the pooled baseline 0.8346 to 0.7636 (`edge_drop` p=0.90, −0.071) and 0.7468 (`node_drop` p=0.90, −0.088), so this backbone demonstrably reads the prerequisite graph. Within the interpretable regime, DDR tracks the AUC drop almost perfectly: the two rise monotonically together (Pearson r=0.95 over the $p \leq 0.3$ core, n=54; Pearson r=0.99 including anchors, n=66; Spearman ρ=0.96 on the same full pool, n=66). Moreover, at a *matched* perturbation budget the operator ordering carries through to accuracy: at p=0.30, `prereq_preserve` costs 0.0098 AUC, `edge_drop` 0.0118, and `node_drop` 0.0423, and the same `prereq_preserve` < `edge_drop` < `node_drop` ordering holds at p=0.10 and p=0.20. The structure-preserving operator that is least disruptive by DDR (Section 4.6) is also the one that costs the least accuracy on a backbone that actually uses the graph.

This is the positive control the earlier DGEKT-only sweep lacked: on a graph-reliant backbone, DDR is not merely a structural audit but a *downstream-predictive* one, and `prereq_preserve`'s structural advantage translates into a measurable accuracy advantage. Because reliance is backbone- and dataset-specific (GKT is graph-reliant on XES3G5M yet inert on ASSISTments), DDR's downstream relevance is likewise conditional: it becomes a design target precisely in the high-reliance regime that the manipulation check identifies. Extending the same anchored protocol to further graph-reliant backbones (e.g. GIKT under the same DDR→AUC protocol) is the natural follow-up (Section 5.4).

4.8 Cold-start KC Diagnostic

Table 14 shows that aggregate AUC can hide per-stratum variation: on real benchmarks the best graph-aware family leads only in *very_cold* on ASSISTments ($\Delta_{\max}$=−0.007) and XES3G5M (+0.006), and is tied or worse in *warm*/*hot* (mean $\Delta_{\max} \approx -0.005$). Junyi *very_cold* and *cold* remain suppressed in the summary (n=11 and 43 on fold 0; fewer than ten discordant label pairs under full pyKT predictions; Tables S12–S13). An A3.5 GPU rerun with uncapped predictions (Server A, 2026-07-25) confirms that models no longer share the earlier capped-path artefact (identical fold 0 AUC for every model); stratum means are still omitted here because fold 0

counts remain too small for stable AUC. Tiny very-cold counts (n<200 on ASSISTments) should not be over-interpreted without fold-level dispersion. Synthetic C2/C5 route all test mass to *hot*; their positive $\Delta_{\max}$ values reflect clean generator structure, not cold-start relief. GKT underperforms sequence leaders on synthetic hot rows, reinforcing model-specific graph benefit. Supplementary Figure S6 (Figure 7) links Table 14 to fold 0 XES3G5M ego-graph and learner-timeline panels (Section J).

Similarity edges and dataset properties.

The similarity-edge rule of Section 3.4 is sensitive to the multi-skill density of the Q-matrix. On the train fold of ASSISTments 2012, 100% of items are tagged with exactly one KC, so $|I(c_i) \cap I(c_j)| = 0$ for every distinct concept pair, and both Jaccard and PMI yield no edges. On XES3G5M, 86.6% of train items are single-skill but the remaining multi-skill items generate 354 similarity edges at the default Jaccard threshold of 0.10. We treat this dataset-property dependency as a first-class outcome of the audit: a similarity rule appropriate for multi-skill benchmarks may need to be replaced or supplemented (e.g., by response-pattern correlation) on single-skill benchmarks. This is not a pipeline failure but a structural observation that protocol papers should report.

4.9 Ground-Truth Cross-Validation on Junyi

Junyi Academy [22] is the only benchmark in our study that ships with a publicly released expert prerequisite annotation (Chang et al. [25]). The annotation enumerates directed exercise-pair relations with a continuous `Prerequisite_avg` score on a mean-opinion-score-like scale. We use $\theta = 0.5$ as the prerequisite filter, retaining 1,131 directed expert edges over 361 distinct exercise endpoints that participate in those edges. Following Section 3.8, expert exercise names are mapped to pipeline `kc_id` via the preprocessing hash function reconstructed from `data/raw/junyi/junyi_Exercise_table.csv`. All 1,131 expert edges align deterministically: the alignment audit reports matched = 1,131, $\text{missing}_{\text{src}} = 0$, $\text{missing}_{\text{dst}} = 0$, and alignment rate 1.0. We compare this expert edge set against the fold 0 train-only inferred list in `e\_pre\_train\_only.csv`, which contains 2,152 ranked directed candidates (the same file audited in Section 4.2; cycle pruning yields 1,139 retained acyclic edges).

Table 15 and Supplementary Figure S7 (Figure 6) show low edge-level overlap (F1 ≈ 0.18 at K=|expert|) that plateaus as K grows, modest direction agreement (≈ 0.26–0.29 on shared pairs), but comparatively high node Jaccard (0.632 at K=|expert|; Supplementary Table S14 reports the full top-K sweep). Reachability F1 rises to 0.339 at large K, indicating recovery of indirect chains despite edge disagreement. Behavioural inference therefore encodes co-occurrence more than pedagogical ordering; it is not a low-cost substitute for expert curation and should be combined as a separate relation when both sources exist. Qualitative disagreement examples appear in Supplementary Section S5. The audit uses θ=0.5 on a single seed and was not swept over stricter thresholds.

5 Discussion

5.1 Model-specific graph benefit under leakage-controlled evaluation

The baseline summary (Table 8) and fold-level diagnostics (Table 9) support a *bounded* benchmarking observation rather than a universal ranking reversal. We anchor the discussion on **XES3G5M** (primary; Section 4.1) because its lower KC-repeat and non-saturated AUC make backbone separation interpretable. Under leakage-controlled evaluation (train-only E_{pre}, optional train-only E_{sim} where populated, learner-based temporal split), explicit concept-graph benefit is *model-specific* within the **primary trio** (*simpleKT*, pyKT GKT, native GIKT) rather than uniform. On XES3G5M, *simpleKT* averages 0.875 AUC while GKT averages 0.834 under the released budget (mean $\Delta\approx-0.041$; 95% CI $[-0.044, -0.038]$, Table S16; fold stds in Table 9; extended-training sensitivity in Section 5.3/Table S21). GIKT (0.878) slightly *exceeds* the sequence leader (+0.003 mean AUC; interval $[+0.003, +0.003]$, tied at three decimals on all folds). Native SKT (0.625) and DyGKT (0.752) are reported as *fork wiring diagnostics* only: their large deficits may reflect under-tuned native stacks rather than inherent backbone limits and are not used to argue that graph-augmented KT fails. On ASSISTments (secondary), the same ordering direction holds but magnitudes shrink (GKT trails by ≈0.007 AUC with near-ceiling leaders). On saturated Junyi, all models cluster within ≈0.002–0.006 AUC of *simpleKT* and should not drive deployment conclusions. Crucially, train-only versus full-log ablation moves headline metrics by at most $|\Delta\text{AUC}|\leq 0.003$ on the three public benchmarks (Table 10), so the protocol's primary *provenance* contribution is reproducibility and risk disclosure, not a demonstrated systematic flip of backbone rankings caused by leakage control alone.

We do not claim that graph-augmented KT is uninformative; the protocol removes a specific channel through which held-out information can reach the model—the concept graph itself. Three non-mutually-exclusive hypotheses are consistent with the observed ordering.

H1 – Leakage in prior evaluations.

A common practice in graph-augmented KT is to compute graph statistics from the full interaction log prior to train/test splitting. Under such practice, graph-augmented models can observe test-fold transitions through the graph during training. Our protocol enforces fold-specific edge inference and provenance logging. Table 10 instantiates the controlled comparison: identical learner splits and evaluation code, but graph statistics pooled over the full preprocessed log. Headline metrics on the three public benchmarks shift by at most $|\Delta\text{AUC}|\leq 0.003$; Synthetic C5 shows a modest shift ($|\Delta\text{AUC}|=0.002$). H1 remains relevant as a *risk hypothesis* about workflows that leak held-out mass into graph construction; it is not a causal claim that we have confirmed on these benchmarks. Table 10 caps how much headline leakage can explain the observed GKT deficit here ($|\Delta\text{AUC}|\leq 0.003$ on public corpora). Architectures that exploit cross-split pooling more aggressively might show larger shifts.

H2 – Aggregate-metric masking.

The cold-start diagnostic (Section 4.8) shows that per-stratum behaviour can differ from aggregate behaviour. Table 14 makes this concrete: on ASSISTments and XES3G5M, the best graph-aware model is slightly better in the *very_cold* stratum, essentially tied in *cold*, and worse in *warm*/*hot*. Junyi *very_cold* and *cold* are omitted for the same reliability reasons as in Section 4.8 (n=11/43 on fold 0; A3.5 full-prediction rerun on GPU). On Synthetic C2/C5, where all test interactions are hot, GIKT/DGEKT improve over sequence-only baselines, but this reflects clean synthetic structure rather than cold-start relief.

H3 – Strong-backbone substitution.

Modern attention-based backbones such as AKT and *simpleKT* may already capture much relational information through long-range temporal attention. Additional structural signal at the concept-embedding level may then yield diminishing returns—a pattern weaker temporal backbones should benefit from more, all else equal.

These hypotheses are not exhaustive. Distinguishing among them requires rerunning prior graph-KT papers under our protocol, which is beyond this paper's scope. The narrower observation is that *under leakage-controlled evaluation on XES3G5M, backbone ordering within the primary trio can differ from near-ceiling aggregates on other corpora*, and benchmarking practice should treat graph provenance and training-budget disclosure as first-order reporting factors—while acknowledging that provenance control alone moved headline AUC by at most 0.003 on the public benchmarks studied here.

Why this strengthens the protocol.

If graph-augmented KT reliably outperformed sequence-only baselines regardless of graph provenance, leakage control would be secondary. The combination of (i) model-specific trio ordering on XES3G5M under train-only graphs, (ii) a near-null train-only/full-log ablation on public benchmarks, (iii) controlled injection showing scalar flags can miss graph-builder contamination (Tables S17–S18), and (iv) edge provenance plus GT cross-validation on Junyi, elevates graph auditing from optional hygiene to a deployment precondition. Combined with Section 4.9, which shows that inferred and expert prerequisite signals are structurally distinct, graph quality cannot be assumed—it must be audited.

Caveats.

Baseline ordering uses three-fold means; DAG audit and GT cross-validation refer to fold 0 exports by default. GKT, AKT, DKT, and *simpleKT* are stock pyKT [17] checkpoints; GIKT, SKT, DyGKT, and DGEKT are native implementations in our fork (`src/models/`). Epoch caps differ in the primary release (30 for sequence checkpoints vs. 10 for graph backbones; Table S15). Extended-training configuration sensitivity for GKT is consolidated in Section 5.3 (Table S21). The GKT–*simpleKT* gap on XES3G5M remains *observational* under attested budgets, not proof that graph signal is useless after exhaustive tuning. The severe SKT/DyGKT deficits in Tables 8 and 9 are flagged as likely implementation-budget artefacts and excluded from primary

ranking claims. Stronger graph-KT architectures [9, 11, 12] may behave differently; re-auditing them under the same protocol is explicit follow-up (Section 5.4).

5.2 Broader implications

Scalability and broader domains.

The protocol is dataset-agnostic: cost scales with train-fold transition counting and capped cycle enumeration, not KT training. Sparsity caps keep audit tractable on million-scale logs [23]. The similarity rule may need domain-specific co-occurrence when Q-matrices are single-skill; GT cross-validation generalises wherever expert DAGs exist; DDR applies to any directed curriculum graph consumed by KT or pretraining pipelines.

Implications for graph-enhanced KT.

The protocol shows that KC graph construction should be treated as a first-class experimental component rather than an unexamined preprocessing step. Fold-specific edge inference, provenance logging, and DAG validation make it possible to identify leakage risks and graph construction choices that would otherwise remain hidden. The DDR results further show that generic graph augmentations can substantially disturb prerequisite structure, especially when they operate at the node level, whereas a prerequisite-preserving operator that protects the transitive-reduction backbone is measurably less disruptive at the same perturbation budget. The anchored downstream stress test in Section 4.7 sharpens this into a *conditional* result. Where a manipulation check confirms the backbone reads the graph (GKT on XES3G5M: destroying the graph costs 0.07–0.09 AUC across three seeds), DDR predicts the accuracy loss (Pearson $r \approx 0.95$ on $p \leq 0.3$, $n=54$; $r \approx 0.99$ with anchors, $n=66$) and `prereq_preserve` costs the least AUC at matched budget—structural damage translates into predictive damage. Where the manipulation check fails (DGEKT, and GKT on ASSISTments: total destruction moves AUC by ≤ 0.003), the same graph is effectively unused and DDR remains a structural/reproducibility audit only. Downstream sensitivity to DDR is therefore backbone- and dataset-specific, matching the reliance factor of the two-factor account; future augmentation papers should report both structural damage (DDR) and a manipulation-check-gated downstream retraining test.

Cold-start reporting.

The cold-start diagnostic shows why KT studies should not rely only on aggregate AUC. Rare knowledge components may behave differently from frequent knowledge components, and apparent gains or high scores on small strata may reflect class imbalance rather than genuine modelling improvement. Reporting per-stratum metrics gives reviewers a clearer view of whether a method is robust across the KC frequency distribution. In our results, graph-aware models show their narrowest real-data advantage in the *very_cold* stratum on ASSISTments and XES3G5M (Junyi *very_cold* suppressed), while synthetic hot-stratum gains reflect clean concept structure rather than cold-start relief. Supplementary Figure S1 links this diagnostic to concrete graph topology on

the public benchmarks; Supplementary Figure S6 gives a fold 0 XES3G5M ego-graph and learner timeline; sparser E_{pre} on ASSISTments and XES3G5M (Supplementary Figure S1) still exhibit cold strata. Very-cold counts remain small (n=49–161 on the reported real-data strata in Table 14); stratum-level AUC therefore carries high variance and should be read alongside n and fold-level dispersion.

Leakage risk concentrates at cold-start KCs.

Leakage control and cold-start stratification are usually treated as separate hygiene checks, but the two-factor model links them directly. *Throughput* is the share of an edge's evidence that comes from held-out mass, and this share is largest exactly where the train-fold neighbourhood is thinnest. A hot KC with hundreds of train transitions barely moves when a few held-out traversals leak into its edges; a very-cold KC supported by only a handful of train transitions can have its incident edges—and hence its graph-propagated representation— dominated by a single leaked transition. Concretely, in the worked example of Section 1 the same 20 leaked traversals that change an edge's support from 40 to 60 (33% throughput) would, on a KC with train support 4, raise it from 4 to 24 (83% throughput): leakage throughput scales inversely with train-fold KC frequency. This is why the leakage taxonomy in Table 1 lists a dedicated *cold-start neighbourhood* class audited by $\text{ECR}^{\text{overlap}}$ together with the strata: cold-start KCs are the highest-throughput entry points for graph-mediated leakage, so per-stratum provenance—not just aggregate provenance—is the conservative report. Equivalently, an aggregate full-log ablation that moves AUC by $\leq$0.003 (Table 10) can still mask a larger, throughput-driven shift confined to the very-cold stratum; disaggregating the ablation by stratum is the natural next measurement.

From audit signal to deployment action.

Table 16 condenses the protocol into a decision map: each audit signal has a reading that triggers a concrete deployment action, independently of headline AUC. This is the operational form of the four-step practitioner workflow in Section 1.

Deployment decision vignettes.

The audit is most actionable when an output maps to a concrete deployment choice:

- **Similarity-edge feasibility (ASSISTments).** Jaccard similarity yields $|E_{\text{sim}}|$=0 under the single-skill Q-matrix (Section 4.8). A practitioner should not deploy a graph model that *requires* similarity edges under this rule without changing the co-occurrence definition or supplying an external relation.
- **Provenance versus headline gain (all public benchmarks).** Train-only versus full-log ablation shifts AUC by at most 0.003 (Table 10). The actionable conclusion is to enforce train-only graph exports for reproducibility and cross-paper comparability, not to expect large accuracy corrections on these corpora.
- **Metric tiering under controlled injection (XES3G5M).** Deliberate test-fold injection increases graph-builder mass while $\text{ECR}_f^{\text{flag}}$=0 (Tables S17–S18). Practitioners should treat fold-disjoint splits as necessary but insufficient and retain provenance logs plus builder-level stress tests before trusting scalar leakage summaries alone.

These vignettes complement—rather than replace—backbone comparisons: C5 reports model-specific trio ordering on XES3G5M under fixed budgets, with Table S21 reporting exploratory configuration sensitivity rather than a compute-matched epoch-only ablation.

Behavioural versus expert prerequisite curation.

The ground-truth cross-validation on Junyi (Section 4.9) shows that train-only behavioural inference and expert prerequisite annotation are correlated at the node level but structurally distinct at the edge level. Two implications follow. First, behavioural inference cannot be treated as a low-cost approximation of expert curation, and papers that report graph-augmented KT results on behaviourally-inferred graphs should not claim those results transfer directly to settings with expert prerequisite graphs (or vice versa). Second, the two signals are complementary: in the most informative setup, a graph-augmented KT model would consume both an inferred behavioural graph and an expert prerequisite graph as separate relations, rather than treating either as a substitute for the other. We do not test this combined setting in the present paper, but the cross-validation provides the quantitative basis for proposing it.

Implications for graph-KT benchmarking.

The bounded observation in Section 5.1 has direct implications for how the graph-KT subfield reports gains. Benchmark tables that omit graph-construction provenance, cold-start strata, and training-budget disclosure are difficult to compare across papers even when aggregate AUC is similar. We do not advocate dismissing reported gains in the graph-KT literature; we advocate *disaggregating* them. A reported AUC improvement of Δ on a benchmark should ideally be decomposed into three components: the part that survives leakage control, the part that depends on graph-construction choices (relation type, threshold, edge cap, encoder), and the part that depends on backbone strength. The protocol in this paper supplies the first component; the cold-start diagnostic supplies a partial view of the second; the third remains an open methodological question.

A practical recommendation follows. When introducing a new graph-augmented KT model, papers should report at minimum: (a) the fold-specific provenance of every retained graph edge; (b) the cold-start stratum on which the gain is most concentrated; (c) the sequence-only baseline matched in compute and hyperparameter budget; (d) when an expert prerequisite annotation is available, the ground-truth cross-validation of the consumed graph against that annotation, using the edge / direction / reachability / node-Jaccard metric family used in Section 4.9; and (e) whether accuracy is actually sensitive to structure-preserving versus structure-damaging graph perturbations. Without these reports, an aggregate-AUC improvement is difficult to attribute. The protocol presented here makes these reports tractable.

5.3 Threats to Validity and Metric Interpretation

Three artefacts invite misreading: the near-null full-log ablation (Table 10), aggressive cycle pruning (Section 4.2), and scalar leakage metrics that are near-degenerate under learner-disjoint splits (Table 5).

The ablation null shift does *not* negate leakage control: graph statistics must not pool held-out learners regardless of backbone sensitivity. Under our filters, pooling rarely changes surviving transitions on public benchmarks; synthetic logs remain more sensitive. We therefore tier audit outputs (Section 3.1): **primary**—provenance logs, $\mathrm{ECR}_f^{\mathrm{flag}}$=0, and GT cross-validation when available; **empirical bounds**—full-log ablation Δ and cold-start strata; **sanity/stress**—$\mathrm{ECR}_f^{\mathrm{overlap}}$, $|\rho_f|$, and TBMR_f. Near-unity overlap and $\mathrm{ECR}_f^{\mathrm{flag}}$=0 are expected under honest learner-disjoint construction and do not replace provenance or GT audits.

Cycle pruning resolves conflicts by dropping the lowest-weight edge per detected cycle (Johnson enumeration, capped at 100 witnesses per round); pruned mass reflects weak or bidirectional transitions rather than arbitrary deletion.

TBMR_f is a within-train temporal stress statistic at a 70% chronological boundary inside each learner's train fold—not cross-split leakage ($\mathrm{ECR}_f^{\mathrm{flag}}$=0). Dense multi-skill co-assessment on XES3G5M and toy-graph structure on Synthetic C2 explain the largest values.

The audit also caught a one-step label misalignment in an early GIKT head (`src/models/gikt.py`), flagged by inconsistent fold AUCs and corrected before reporting Table 9.

Limitations.

Three-fold CV limits inferential power of fold-level tests ($p_{\min}$=0.25 for Wilcoxon[1]). Primary XES3G5M ranking claims rely on fold-level ΔAUC magnitudes (Table 9) and Table S16 intervals (`scripts/bootstrap_auc_ci.py`; paired-t over three folds by default, matching fold-level ΔAUC; optional `--learner-bootstrap` on exported parquets). The GKT–*simpleKT* gap is *observational* under attested maximum epoch budgets (30 vs. 10; Table S15). Due to the substantially greater computational cost of GKT, the primary GKT configuration used a maximum of 10 epochs, whereas *simpleKT* used a maximum of 30 epochs. A targeted seed-42 three-fold check evaluated an extended GKT configuration with a maximum of 30 epochs; its batch size, hidden dimension, and maximum sequence length also differed from the primary configuration (batch 4 vs. batch 32, and smaller hidden/sequence caps). The observed mean AUC was 0.003451 higher under the extended-training configuration, but the approximate 95% CI $[-0.004, 0.011]$ included zero. Because multiple hyperparameters changed together, this result should be interpreted as exploratory configuration sensitivity rather than an isolated epoch effect. The study does not provide a fully compute-matched comparison. Nine-fold GKT30 artefacts at batch 32 (Server A attestation) yield a pooled ΔAUC of ≈ -0.038 versus *simpleKT* under 30-epoch budgets—directionally consistent with the primary -0.041 gap but still not batch-matched. Native SKT/DyGKT runs are wiring diagnostics, not tuned-backbone benchmarks. Controlled injection (Section 4.3, Tables S17–S18) demonstrates metric-level sensitivity. ASSISTments yields no E_{sim} under item-overlap Jaccard; GT cross-validation is Junyi-only at θ=0.5; pyKT GKT on Junyi Academy is omitted because dense multi-million interaction graphs make the stock GKT wrapper impractical under our

[1] With three learner-disjoint folds, the two-sided Wilcoxon signed-rank test has minimum attainable p=0.250.

compute budget (Tables S1, 8); downstream DDR retraining covers DGEKT and GKT (the latter with a manipulation-check anchor), with GKT/XES3G5M reported across three seeds ($\{42, 17, 1234\}$, nine folds).

Scope of the two-factor claim.

Two limits bound the conditional-harm framing of Table 7. First, the high-throughput cell is reached only by *controlled* injection; we have not exhibited a natural workflow (absent deliberate contamination) that raises throughput on these corpora, so the dangerous cell is demonstrated, not shown to be common. Closing this gap would require a realistic mechanism that lifts throughput without hand injection—a looser but commonly used support filter, a denser-graph corpus, or a re-built legacy graph-KT pipeline. Second, the reliance factor currently rests on GKT and GIKT (positive) versus *simpleKT* and DGEKT (≈ 0); the interaction is an illustration backed by two graph-reliant backbones rather than an exhaustive characterisation. A matched DDR$\rightarrow$AUC sweep on additional graph-reliant backbones at both throughput levels would upgrade the interaction from illustration to a tested claim.

5.4 Future work

Three extensions follow naturally from the audit boundary established here. **Continuous and online graph updating.** Deployed tutoring systems accumulate new interactions over time; G_f should be refreshable from post-deployment logs under explicit temporal cutoffs so that graph updates do not re-pool held-out or future learners. An online variant would extend the tiered leakage diagnostics (ECR, TBMR) to sliding windows and report when structural drift warrants retraining versus graph-only patches. **Feedback-aware graph construction.** Behavioural prerequisite inference currently weights co-occurrence and temporal precedence. Future work can condition edge retention on learning gain, hint usage, or remediation outcomes while keeping the train-only provenance contract; the edge–outcome diagnostic $|\rho|$ (Eq. (3)) provides a starting point for detecting target-conditional artefacts. **Broader backbone and operator coverage.** Re-auditing additional graph consumers under the same protocol, and testing hierarchy-respecting augmentations beyond DDR, would clarify whether structural diagnostics transfer across encoders. A fully compute-matched GKT vs. *simpleKT* comparison (matched maximum epochs *and* batch size on the same folds) remains open; exploratory GKT30 runs at batch 32 (Table S21 context) narrow but do not close this gap. **DDR downstream coverage.** The anchored retraining sweep now covers two graph consumers (DGEKT and GKT), including a completed three-seed GKT replication on XES3G5M, and establishes the manipulation-check protocol that separates graph-reliant from graph-inert backbones. Remaining steps are to add a second graph-reliant backbone at both throughput levels (e.g. GIKT), and to pair the edge-level DDR correlation with the reachability-disruption variant so that *which* structural property a backbone consumes can be identified, not just that it consumes one.

6 Conclusion

We presented a leakage-controlled audit protocol for constructing and evaluating concept graphs in graph-enhanced KT. The contribution is framed for *applied practitioners* as a *decision-support and risk-scoring layer* for trustworthy deployment: a practitioner checklist, provenance artefacts, tiered leakage diagnostics, DDR augmentation metrics, ground-truth cross-validation where expert graphs exist, and cold-start KC stratification. On XES3G5M (primary), ASSISTments (secondary), and Junyi (saturated sanity), the audit produces acyclic train-only prerequisite graphs after transparent cycle pruning. An anchored downstream retraining test makes the predictive reading conditional: on a graph-reliant backbone that passes a manipulation check (GKT on XES3G5M; three seeds) DDR predicts AUC loss (Pearson $r \approx 0.95$ on $p \leq 0.3$, $n=54$) and `prereq_preserve` costs the least accuracy at matched budget, whereas on graph-inert cells (DGEKT; GKT on ASSISTments) total graph destruction moves AUC by ≤ 0.003.

Under fixed training budgets on XES3G5M, the primary trio shows a *model-specific* pattern: GKT trails *simpleKT* by ≈ 0.041 mean AUC (95% CI $[-0.044, -0.038]$; Table S16) while GIKT remains competitive. Train-only versus full-log ablation moves headline AUC by at most 0.003. These results cohere under a two-factor reading: graph-mediated leakage shifts AUC only when contamination throughput and backbone reliance are jointly high; under the verified injection sweep, fold-0 AUC nonetheless stays near flat (Table S18) even while builder mass rises (Table S17). Practitioners can apply the five-item checklist in Section 5.2 before reporting new graph-augmented KT results. Section 5.4 outlines continuous graph updating and feedback-aware construction as next steps. Reproduction artefacts are at `https://github.com/edu-risk-lab/leakage-controlled-kt-audit.git`.

Appendix overview

Tables. S1–S5: per-dataset baseline diagnostics (Section C); S6–S10: per-dataset graph ablation (Section D); S11: paired significance tests on public benchmarks (Section E); S12–S13: cold-start diagnostics (Section J); S14: ground-truth metrics including node Jaccard (Section I); S15: training-configuration disclosure (Section M); S16: ΔAUC intervals for primary pairs (Section N); S17–S18: controlled leak injection and downstream AUC (Section P); S19–S20: exploratory ANOVA on backbone and DGEKT downstream AUC (Section Q; Tables 37–38); S21: targeted three-fold extended-training configuration-sensitivity check, seed 42 (Section O).

Sections and figures. Section S3: sequence-autocorrelation analysis (Section F; Table 28, Figure 5); Section S4: DDR augmentation operators (Section G); Section S5: ground-truth disagreement examples (Section H); Figure S1: multi-benchmark train-only E_{pre} samples (Figure 9); Figure S2: multi-benchmark DDR curves (Figure 10); Figure S3: DDR downstream scatter plot (Figure 8); Figure S4: end-to-end protocol plumbing (Figure 3); Figure S5: train-only prerequisite construction pipeline (Figure 4); Figure S6: cold-start ego-graph and learner timeline (Figure 7); Figure S7: Junyi ground-truth precision–recall curve (Figure 6). Cold-start and GT panels

are generated from fold 0 exports by `scripts/plot\_kt\_graph\_figures.py`. Section S2: formal graph-construction objective (Section A); protocol plumbing figures (Section B).

A Formal graph-construction objective

This section states the conceptual graph objective referenced in Section 3.1. Prerequisite and similarity edges are inferred in separate greedy pipelines with λ=0 in practice; sim is Jaccard or PMI (Section 3.4).

$$\max_{E^f_{\text{pre}}, E^f_{\text{sim}}} \sum_{(i,j)\in E^f_{\text{pre}}} w_{ij} \;+\; \lambda \sum_{(i,j)\in E^f_{\text{sim}}} \text{sim}(c_i, c_j) \tag{8}$$

where $\lambda \geq 0$ would trade prerequisite mass against similarity mass if the two pipelines were merged (they are not in the released implementation).

$$\text{(C1) } \forall e \in \mathcal{E}_f,\ S_{\text{hi}}(e) = \emptyset; \tag{9}$$

$$\begin{aligned}\text{(C2) } &\mathcal{U}^{\text{tr}}_f \cap \mathcal{U}^{\text{va}}_f = \emptyset,\\ &\mathcal{U}^{\text{tr}}_f \cap \mathcal{U}^{\text{te}}_f = \emptyset,\\ &\mathcal{U}^{\text{va}}_f \cap \mathcal{U}^{\text{te}}_f = \emptyset;\end{aligned} \tag{10}$$

$$\text{(C3) } E^f_{\text{pre}} \text{ is acyclic after pruning;} \tag{11}$$

$$\begin{aligned}\text{(C4) } &|E^f_{\text{pre}}| \leq K,\ |\text{Out}_{E^f_{\text{pre}}}(i)| \leq k;\\ &\text{sim}(c_i, c_j) \geq \tau\ \forall (i,j) \in E^f_{\text{sim}}.\end{aligned} \tag{12}$$

B Protocol plumbing figures

C Per-dataset baseline diagnostics

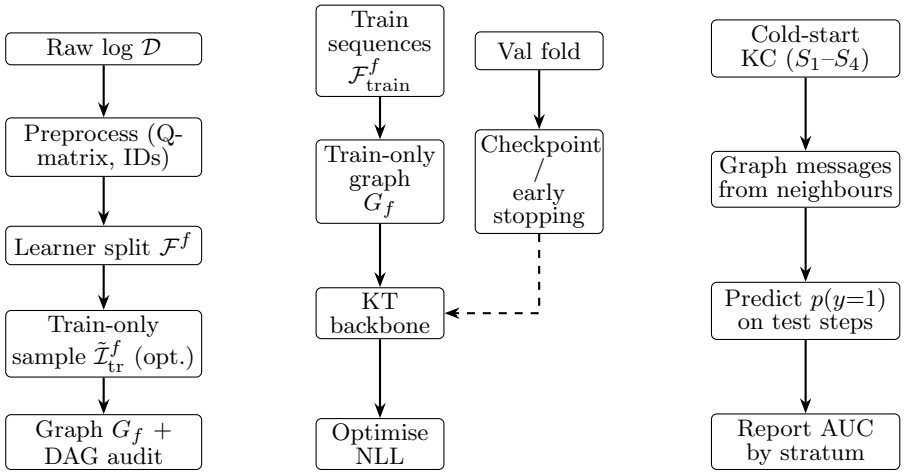
Tables S1–S5 (Tables 17–21) reproduce the five diagnostic baseline tables (AUC, ACC, NLL with 95% bootstrap CIs) that the main text summarises in its cross-dataset AUC summary table. pyKT GKT is omitted on Junyi Academy because the stock wrapper is impractical on the dense multi-million-interaction graphs under our compute budget.

D Per-dataset graph construction ablation

Tables S6–S10 (Table 22–26) report fold 0 train-only versus full-log metrics for each graph-augmented pyKT model. The main text reports only the maximum |ΔAUC— and |ΔACC per dataset.

E Paired significance tests (public benchmarks)

Table S11 (Table 27) reports paired t-test and Wilcoxon p-values for graph-augmented models versus *simpleKT* on the three public benchmarks. The main text baseline-CV



(a) Data pipeline from raw log to audited G_f.

(b) Training inputs and validation-based checkpoint selection.

(c) Cold-start inference and stratum reporting.

Fig. 3: Supplementary Figure S4. End-to-end protocol plumbing: (a) data preparation and fold-conditioned graph construction, (b) training with a train-only graph and validation-based checkpoint selection, and (c) cold-start knowledge-component evaluation with stratum-specific reporting (Section 4.8).

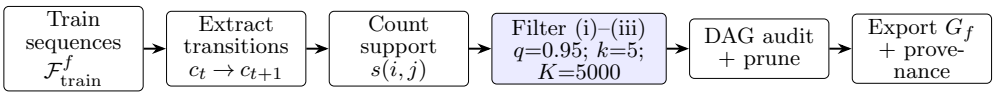


Fig. 4: Supplementary Figure S5. Train-only prerequisite construction pipeline: temporal transitions are counted on $\mathcal{F}^f_{\text{train}}$ (or optional sample $\tilde{\mathcal{I}}^f_{\text{tr}}$), filtered, cycle-pruned, and exported with provenance fields (Section 3.5).

table lists only Wilcoxon p-values because three-fold CV limits Wilcoxon resolution ($p_{\min}$=0.25).

F Sequence autocorrelation analysis

For every learner we order interactions by timestamp and, over consecutive within-learner pairs $(t{-}1, t)$, measure KC-repeat rate, item-repeat rate, label persistence, lag-one label autocorrelation ρ, and the AUC of the trivial previous-answer predictor (`scripts/compute_autocorrelation.py`). Table 28 reports the scalar diagnostics; Figure 5 plots consecutive KC-repeat rate against *simpleKT* (and BKT) AUC across the three public benchmarks.

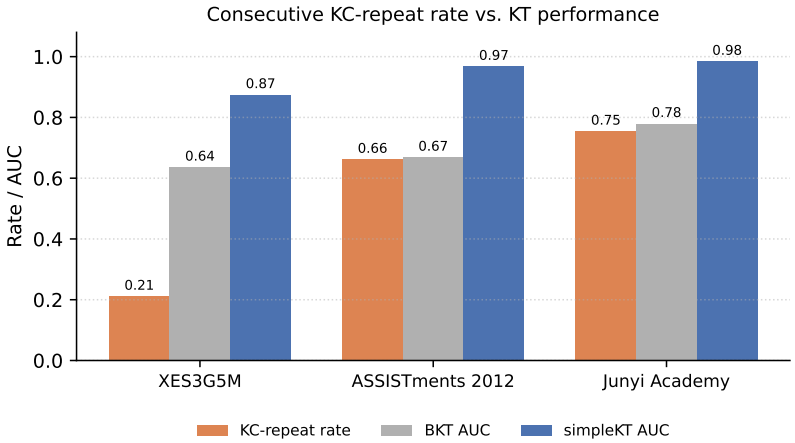


Fig. 5: Consecutive KC-repeat rate against *simpleKT* AUC across the three public benchmarks.

G DDR augmentation operator definitions

All operators act on the retained acyclic E_{pre} at perturbation strength $p \in \{0.05, 0.10, 0.20, 0.30\}$:

- **Attribute masking** masks a random subset of edge attributes without changing E_{pre} (DDR=0 by construction).
- **Edge dropping** removes each edge independently with probability p (expected DDR$\approx p$).
- **Subgraph sampling** performs a random walk until $\lceil (1-p)|\mathcal{C}| \rceil$ nodes are visited and returns the induced subgraph (boundary edges drop; internal connectivity preserved).
- **Node dropping** removes $\lceil p|\mathcal{C}| \rceil$ nodes and all incident edges.
- `prereq_preserve` draws the same per-edge budget as edge dropping but removes only transitively redundant edges, protecting the transitive-reduction backbone.

H Ground-truth disagreement examples (Junyi)

The cross-validation procedure exports qualitative disagreement cases to `disagreement\_examples.csv` (repository tables/). Inferred-only edges are predominantly bidirectional co-occurring arithmetic pairs (e.g. `number_line` $\leftrightarrow$ `addition_1`). Expert-only edges are systematic level-progression annotations (e.g. `equivalent_fractions_2` $\rightarrow$ `equivalent_fractions`). The `wrong_direction` category contains many reversed level-progression pairs such as `addition_1` $\rightarrow$ `addition_2`.

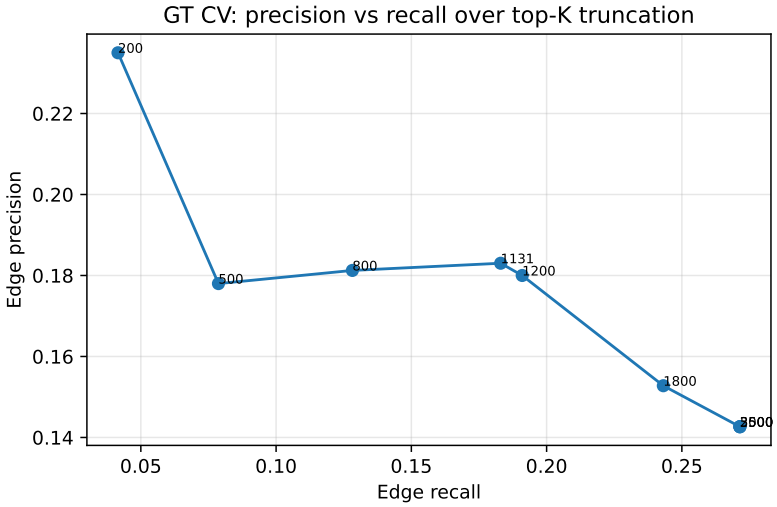


Fig. 6: Supplementary Figure S7. Edge-level precision and recall of train-only inferred E_{pre} versus the expert prerequisite annotation on Junyi, across top-K cutoffs $K \in \{200, 500, 800, 1131, 1200, 1800, 2500, 3500, 5000\}$. The curve plateaus at $K \geq$ 2,500 because the inferred edge pool tops out at 2,152 after DAG audit.

I Ground-truth metrics with node Jaccard

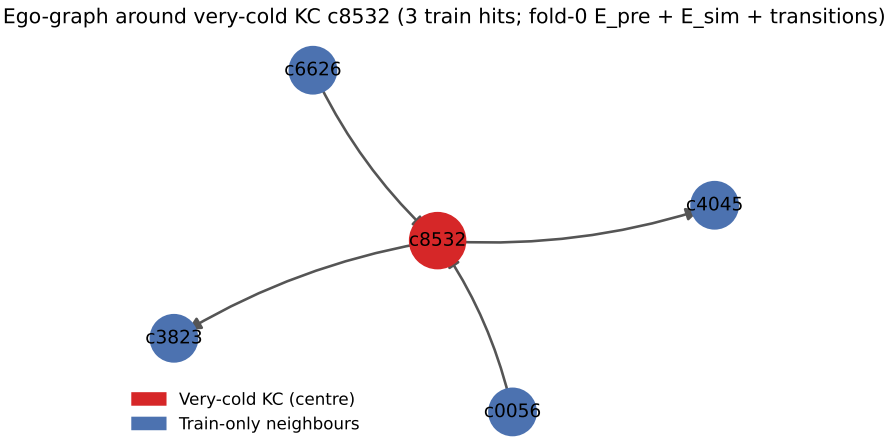
Table S14 (Table 29) extends the main text ground-truth table with node Jaccard across the same top-K cutoffs.

J Cold-start tables (full)

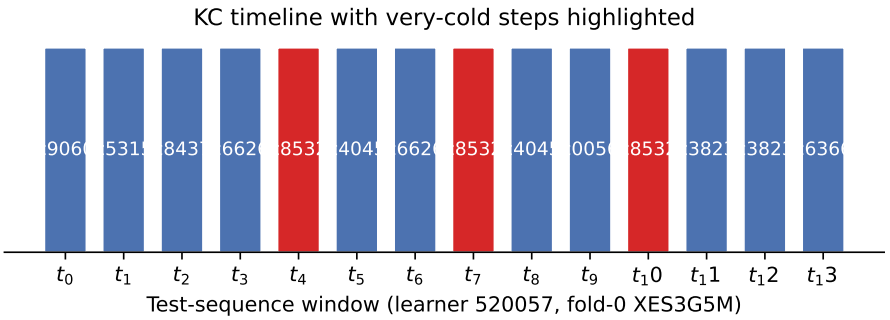
Table S12 (Table 30) reports fold 0 *simpleKT* metrics by KC-frequency stratum; Table S13 (Table 31) lists three-fold per-model AUC (mean $\pm$ std) and $\Delta_{\max}$ by stratum. The main text summarises both in a single table.

K DDR downstream scatter plot

L Additional KT-graph figures



(a) Cold-start KC ego-graph (fold 0 export).



(b) Learner KC timeline with cold steps.

Fig. 7: Supplementary Figure S6. Cold-start visualisation on XES3G5M (fold 0; very-cold stratum n=161 validation+test held-out interactions for *simpleKT* in Table 14). Panels are generated from released fold exports and the preprocessed interaction log by `scripts/plot\_kt\_graph\_figures.py`. **(a)** Ego-graph around a very-cold KC (red): neighbours combine directed edges from E_{pre}, E_{sim} where present, and train-only temporal transitions. **(b)** A test-sequence window for one held-out learner with repeated very-cold steps; stratum labels derive from train-fold counts only (Section 3.7).

Table 12: DDR vs. downstream graph-KT accuracy. For each dataset and model, the prerequisite graph E_{pre} is perturbed by each operator at strength p; *AUC drop* is the mean decrease in test AUC relative to the unperturbed (DDR= 0) baseline graph across folds. Per-(dataset, model) correlations are distinct estimands: assist2012/dgekt: Pearson r=0.15 (n=27), Spearman ρ=0.18; assist2012/gkt: Pearson r=0.97 (n=33), Spearman ρ=0.95; xes3g5m/dgekt: Pearson r=0.13 (n=27), Spearman ρ=0.18; xes3g5m/gkt: Pearson r=0.99 (n=99), Spearman ρ=0.96 The global pooled Pearson across both datasets × both models is annotated on Fig. S3 (n=186) and must not be conflated with XES3G5M/GKT-only figures.

Dataset	Model	Operator	p	Mean DDR	Mean AUC	AUC drop
assist2012	dgekt	edge_drop	0.10	0.086	0.9600	0.0000
assist2012	dgekt	edge_drop	0.20	0.221	0.9601	-0.0000
assist2012	dgekt	edge_drop	0.30	0.304	0.9600	-0.0000
assist2012	dgekt	node_drop	0.10	0.171	0.9601	-0.0001
assist2012	dgekt	node_drop	0.20	0.369	0.9601	-0.0000
assist2012	dgekt	node_drop	0.30	0.493	0.9601	-0.0000
assist2012	dgekt	prereq_preserve	0.10	0.080	0.9601	-0.0001
assist2012	dgekt	prereq_preserve	0.20	0.206	0.9601	-0.0000
assist2012	dgekt	prereq_preserve	0.30	0.278	0.9600	0.0000
assist2012	gkt	edge_drop	0.10	0.086	0.9630	-0.0000
assist2012	gkt	edge_drop	0.20	0.221	0.9629	0.0002
assist2012	gkt	edge_drop	0.30	0.304	0.9627	0.0003
assist2012	gkt	edge_drop	0.90	0.899	0.9609	0.0018
assist2012	gkt	node_drop	0.10	0.199	0.9626	0.0004
assist2012	gkt	node_drop	0.20	0.460	0.9619	0.0012
assist2012	gkt	node_drop	0.30	0.553	0.9619	0.0011
assist2012	gkt	node_drop	0.90	0.986	0.9607	0.0021
assist2012	gkt	prereq_preserve	0.10	0.075	0.9630	0.0000
assist2012	gkt	prereq_preserve	0.20	0.187	0.9629	0.0001
assist2012	gkt	prereq_preserve	0.30	0.255	0.9628	0.0003
xes3g5m	dgekt	edge_drop	0.10	0.093	0.8417	0.0001
xes3g5m	dgekt	edge_drop	0.20	0.214	0.8416	0.0001
xes3g5m	dgekt	edge_drop	0.30	0.301	0.8417	0.0000
xes3g5m	dgekt	node_drop	0.10	0.177	0.8417	0.0001
xes3g5m	dgekt	node_drop	0.20	0.424	0.8414	0.0003
xes3g5m	dgekt	node_drop	0.30	0.555	0.8417	0.0001
xes3g5m	dgekt	prereq_preserve	0.10	0.071	0.8417	0.0001
xes3g5m	dgekt	prereq_preserve	0.20	0.168	0.8417	0.0001
xes3g5m	dgekt	prereq_preserve	0.30	0.233	0.8415	0.0003
xes3g5m	gkt	edge_drop	0.10	0.095	0.8313	0.0033
xes3g5m	gkt	edge_drop	0.20	0.202	0.8272	0.0074
xes3g5m	gkt	edge_drop	0.30	0.298	0.8221	0.0124
xes3g5m	gkt	edge_drop	0.90	0.901	0.7637	0.0708
xes3g5m	gkt	node_drop	0.10	0.189	0.8211	0.0135
xes3g5m	gkt	node_drop	0.20	0.381	0.8058	0.0287
xes3g5m	gkt	node_drop	0.30	0.534	0.7928	0.0418
xes3g5m	gkt	node_drop	0.90	0.990	0.7469	0.0877
xes3g5m	gkt	prereq_preserve	0.10	0.073	0.8321	0.0025
xes3g5m	gkt	prereq_preserve	0.20	0.160	0.8285	0.0060
xes3g5m	gkt	prereq_preserve	0.30	0.237	0.8250	0.0096

Table 13: DDR→downstream AUC for the graph-reliant backbone GKT, with a positive control. For each dataset the prerequisite graph E_{pre} is perturbed by each operator at strength p; *AUC drop* is the mean decrease in test AUC relative to the unperturbed (DDR= 0) baseline across folds (baselines: XES3G5M 0.8346, ASSISTments 0.9630; XES3G5M pooled over 3 seeds × 3 folds). The bottom block reports the manipulation-check anchors (p=0.90, near-total graph destruction). On XES3G5M/GKT, DDR tracks AUC drop with distinct estimands: Pearson r=0.95 on the $p \leq 0.3$ core (n=54); Pearson r=0.99 on the full pool including anchors (n=66); Spearman ρ=0.96 on that same full pool (n=66). These are not interchangeable with the global pooled Pearson $r \approx 0.75$ (n=186) annotated on Fig. S3. At matched budget, `prereq_preserve` degrades AUC least and `node_drop` most. On ASSISTments the same total destruction moves GKT by ≤0.003, so this cell (like DGEKT in Table 12) is a *low-reliance anchor* whose correlation is not practically meaningful.

Dataset	Operator	p	Mean DDR	Mean AUC	AUC drop
XES3G5M	`edge_drop`	0.10	0.096	0.8312	0.0032
XES3G5M	`edge_drop`	0.20	0.206	0.8270	0.0075
XES3G5M	`edge_drop`	0.30	0.299	0.8226	0.0118
XES3G5M	`node_drop`	0.10	0.193	0.8210	0.0134
XES3G5M	`node_drop`	0.20	0.397	0.8051	0.0293
XES3G5M	`node_drop`	0.30	0.551	0.7921	0.0423
XES3G5M	`prereq_preserve`	0.10	0.076	0.8317	0.0027
XES3G5M	`prereq_preserve`	0.20	0.168	0.8274	0.0070
XES3G5M	`prereq_preserve`	0.30	0.242	0.8246	0.0098
ASSIST2012	`edge_drop`	0.10	0.086	0.9630	-0.0000
ASSIST2012	`edge_drop`	0.20	0.221	0.9629	0.0002
ASSIST2012	`edge_drop`	0.30	0.304	0.9627	0.0003
ASSIST2012	`node_drop`	0.10	0.199	0.9626	0.0004
ASSIST2012	`node_drop`	0.20	0.460	0.9619	0.0012
ASSIST2012	`node_drop`	0.30	0.553	0.9619	0.0011
ASSIST2012	`prereq_preserve`	0.10	0.075	0.9630	0.0000
ASSIST2012	`prereq_preserve`	0.20	0.187	0.9629	0.0001
ASSIST2012	`prereq_preserve`	0.30	0.255	0.9628	0.0003
Manipulation-check anchors (near-total destruction)					
XES3G5M	`edge_drop`	0.90	0.901	0.7636	0.0708
XES3G5M	`node_drop`	0.90	0.991	0.7468	0.0876
ASSIST2012	`edge_drop`	0.90	0.906	0.9612	0.0018
ASSIST2012	`node_drop`	0.90	0.995	0.9610	0.0021

Table 14: Cold-start KC diagnostic summary. Column n counts fold 0 validation+test held-out interactions per KC-frequency stratum (strata assigned from train-fold counts only; *simpleKT*); *simpleKT* AUC and $\Delta_{\max}$ report three-fold means ± std ($\Delta_{\max}$: best graph-family AUC minus best sequence-only AUC). Cells marked "—" are suppressed when fold 0 stratum counts are too small or stratum AUC is undefined/unreliable (see Supplementary Tables S12–S13). Per-model strata appear in Supplementary Tables S12–S13.

Dataset	Stratum	n	*simpleKT* AUC	$\Delta_{\max}$
Junyi Academy	very_cold	11	—	—
	cold	43	—	—
	warm	2,320	0.846 ± 0.016	$+0.005 \pm 0.003$
	hot	$\approx 7.7 \times 10^6$	0.982 ± 0.000	$+0.000 \pm 0.000$
ASSIST'12	very_cold	49	0.724 ± 0.100	-0.007 ± 0.012
	cold	707	0.772 ± 0.003	$+0.000 \pm 0.000$
	warm	9,017	0.756 ± 0.017	$+0.001 \pm 0.000$
	hot	$\approx 8.1 \times 10^5$	0.696 ± 0.002	-0.005 ± 0.000
XES3G5M	very_cold	161	0.817 ± 0.013	$+0.006 \pm 0.006$
	cold	2,592	0.804 ± 0.007	-0.000 ± 0.001
	warm	17,655	0.787 ± 0.003	-0.007 ± 0.000
	hot	$\approx 1.9 \times 10^6$	0.717 ± 0.000	-0.009 ± 0.000
Synth C2	hot	60,000	0.626 ± 0.002	$+0.008 \pm 0.000$
Synth C5	hot	60,000	0.630 ± 0.003	$+0.002 \pm 0.001$

Table 15: GT cross-validation: agreement between train-only inferred E_{pre} and expert prerequisite DAG on Junyi Academy ($|E| = 1131$ directed edges).

K (top inferred)	Edge prec.	Edge rec.	Dir. agr.	Rch. F1
200	0.235	0.042	0.346	0.008
500	0.178	0.079	0.225	0.021
800	0.181	0.128	0.280	0.062
$\lvert E_{\text{expert}}\rvert = 1131$	0.183	0.183	0.261	0.104
1200	0.180	0.191	0.262	0.126
1800	0.153	0.243	0.278	0.260
2500	0.143	0.271	0.285	0.339
3500	0.143	0.271	0.285	0.339
5000 (cap)	0.143	0.271	0.285	0.339

Table 16: Decision map: how each audit signal maps to a deployment action, independently of headline accuracy. Thresholds are the operating points used in this paper; teams should recalibrate per corpus.

Audit signal	Reading	Recommended action
Builder mass / TBMR_f (throughput)	Rises with pooled vs. train-only support	Enforce train-only G_f; distrust any AUC gain in the high-throughput regime
$\mathrm{ECR}_f^{\mathrm{flag}}$ (learner-disjoint)	Can stay 0 while throughput is high (Tables S17–S18)	Treat fold-disjoint split as necessary but *insufficient*; keep provenance logs
Manipulation check (p=0.90 anchor)	AUC unmoved under near-total graph destruction	Backbone is graph-inert; report DDR as reproducibility scaffolding, not a design target
DDR vs. AUC (graph-reliant backbone)	Monotone on GKT/XES3G5M (Pearson $r\approx0.95$, n=54 core; $r\approx0.99$, n=66 with anchors)	Prefer `prereq_preserve`; DDR is a valid design target here
Similarity-edge feasibility	$\lvert E_{\mathrm{sim}}\rvert$=0 under single-skill Q-matrix	Do not deploy a model *requiring* E_{sim} without a new co-occurrence rule
Cold-start stratum (n, throughput)	Highest leakage throughput at thinnest train support	Report per-stratum provenance; audit $\mathrm{ECR}^{\mathrm{overlap}}$ on very-cold knowledge components

Table 17: Diagnostic baseline results (train-only graphs). Baselines are used for protocol comparison only; no SOTA claim is made. Values are fold means with 95% bootstrap confidence intervals over valid+test sequence positions (`n_folds` as exported per dataset).

Model	AUC	ACC	NLL
AKT	0.984 [0.982, 0.989]	0.951 [0.945, 0.962]	0.121 [0.093, 0.135]
BKT	0.780 [0.779, 0.781]	0.846 [0.845, 0.847]	0.379 [0.377, 0.380]
DGEKT	0.981 [0.981, 0.981]	0.943 [0.942, 0.943]	0.137 [0.136, 0.138]
DKT	0.984 [0.982, 0.989]	0.951 [0.945, 0.962]	0.120 [0.093, 0.134]
DyGKT	0.978 [0.978, 0.978]	0.934 [0.933, 0.934]	0.151 [0.151, 0.152]
GIKT	0.982 [0.982, 0.982]	0.946 [0.946, 0.946]	0.133 [0.132, 0.133]
simpleKT	0.984 [0.982, 0.989]	0.951 [0.945, 0.962]	0.121 [0.093, 0.135]
SKT	0.979 [0.976, 0.986]	0.938 [0.931, 0.954]	0.139 [0.107, 0.156]

Table 18: Diagnostic baselines: ASSISTments 2012 (three-fold learner CV (`n_folds`=3); train-only graphs).

Model	AUC	ACC	NLL
AKT	0.968 [0.968, 0.969]	0.916 [0.914, 0.917]	0.175 [0.173, 0.176]
BKT	0.669 [0.667, 0.671]	0.713 [0.711, 0.715]	0.574 [0.573, 0.576]
DGEKT	0.960 [0.959, 0.961]	0.907 [0.906, 0.908]	0.186 [0.185, 0.188]
DKT	0.963 [0.961, 0.964]	0.910 [0.908, 0.911]	0.182 [0.180, 0.185]
DyGKT	0.952 [0.951, 0.952]	0.899 [0.898, 0.900]	0.200 [0.198, 0.203]
GIKT	0.967 [0.967, 0.968]	0.915 [0.914, 0.915]	0.175 [0.174, 0.177]
GKT	0.961 [0.960, 0.962]	0.907 [0.906, 0.908]	0.185 [0.184, 0.187]
simpleKT	0.968 [0.967, 0.968]	0.916 [0.915, 0.916]	0.173 [0.172, 0.175]
SKT	0.953 [0.952, 0.953]	0.898 [0.897, 0.899]	0.198 [0.196, 0.200]

Table 19: Diagnostic baselines: XES3G5M (three-fold learner CV (`n_folds`=3); train-only graphs).

Model	AUC	ACC	NLL
AKT	0.875 [0.874, 0.875]	0.858 [0.857, 0.858]	0.324 [0.323, 0.326]
BKT	0.636 [0.634, 0.638]	0.793 [0.791, 0.793]	0.490 [0.490, 0.492]
DGEKT	0.841 [0.841, 0.842]	0.844 [0.844, 0.845]	0.358 [0.356, 0.358]
DKT	0.858 [0.857, 0.858]	0.851 [0.851, 0.852]	0.342 [0.341, 0.342]
DyGKT	0.752 [0.752, 0.754]	0.813 [0.813, 0.814]	0.431 [0.429, 0.431]
GIKT	0.878 [0.878, 0.878]	0.860 [0.859, 0.860]	0.321 [0.320, 0.322]
GKT	0.834 [0.833, 0.835]	0.842 [0.841, 0.842]	0.364 [0.363, 0.365]
simpleKT	0.875 [0.874, 0.875]	0.858 [0.857, 0.859]	0.324 [0.323, 0.325]
SKT	0.625 [0.623, 0.626]	0.800 [0.799, 0.801]	0.482 [0.480, 0.483]

Table 20: Diagnostic baselines: Synthetic C2 (three-fold learner CV (`n_folds`=3); train-only graphs).

Model	AUC	ACC	NLL
AKT	0.873 [0.870, 0.876]	0.808 [0.802, 0.812]	0.407 [0.401, 0.417]
BKT	0.835 [0.833, 0.838]	0.782 [0.780, 0.784]	0.460 [0.458, 0.462]
DGEKT	0.710 [0.634, 0.859]	0.724 [0.686, 0.794]	0.551 [0.432, 0.613]
DKT	0.862 [0.857, 0.868]	0.798 [0.788, 0.805]	0.421 [0.413, 0.434]
DyGKT	0.710 [0.631, 0.867]	0.726 [0.686, 0.801]	0.550 [0.421, 0.617]
GIKT	0.875 [0.871, 0.880]	0.809 [0.802, 0.815]	0.405 [0.398, 0.416]
GKT	0.873 [0.868, 0.878]	0.808 [0.799, 0.814]	0.407 [0.400, 0.420]
simpleKT	0.875 [0.871, 0.880]	0.809 [0.803, 0.814]	0.405 [0.399, 0.416]
SKT	0.708 [0.634, 0.853]	0.722 [0.686, 0.788]	0.554 [0.440, 0.613]

Table 21: Diagnostic baselines: Synthetic C5 (three-fold learner CV (`n_folds`=3); train-only graphs).

Model	AUC	ACC	NLL
AKT	0.822 [0.819, 0.823]	0.750 [0.746, 0.752]	0.500 [0.496, 0.507]
BKT	0.791 [0.789, 0.794]	0.724 [0.722, 0.726]	0.534 [0.532, 0.536]
DGEKT	0.830 [0.827, 0.832]	0.756 [0.755, 0.757]	0.489 [0.486, 0.492]
DKT	0.826 [0.825, 0.827]	0.754 [0.752, 0.756]	0.493 [0.491, 0.496]
DyGKT	0.826 [0.824, 0.828]	0.751 [0.748, 0.753]	0.492 [0.490, 0.496]
GIKT	0.820 [0.817, 0.824]	0.749 [0.744, 0.754]	0.500 [0.496, 0.504]
GKT	0.810 [0.806, 0.814]	0.738 [0.730, 0.744]	0.526 [0.513, 0.538]
simpleKT	0.829 [0.828, 0.831]	0.755 [0.754, 0.757]	0.491 [0.489, 0.492]
SKT	0.810 [0.808, 0.813]	0.740 [0.737, 0.744]	0.511 [0.507, 0.513]

Table 22: Ablation contrasting *train-only* graph smoothing (protocol) with a *full-log* graph built from all learners before splitting (leaky construction). Graph-augmented rows are pyKT checkpoints consuming exported adjacency from P0. Δ is fold 0 full-log minus train-only. *to*: train-only graph; *fl*: full-log graph.

Model	AUC to	AUC fl	ΔAUC	ACC to	ACC fl	ΔACC
GIKT	0.982	0.982	0.000	0.946	0.946	-0.000
SKT	0.979	0.976	-0.003	0.938	0.931	-0.008
DyGKT	0.978	0.978	-0.000	0.934	0.934	0.000
DGEKT	0.981	0.981	-0.000	0.943	0.943	-0.000

Table 23: Train-only vs. full-log graph ablation: ASSISTments 2012.

Model	AUC to	AUC fl	ΔAUC	ACC to	ACC fl	ΔACC
GKT	0.961	0.961	-0.000	0.907	0.907	-0.000
GIKT	0.967	0.968	0.000	0.915	0.915	0.000
SKT	0.953	0.953	0.000	0.898	0.898	0.000
DyGKT	0.952	0.952	-0.000	0.899	0.899	-0.000
DGEKT	0.960	0.960	0.000	0.907	0.907	0.000

Table 24: Train-only vs. full-log graph ablation: XES3G5M.

Model	AUC to	AUC fl	ΔAUC	ACC to	ACC fl	ΔACC
GKT	0.834	0.835	0.002	0.842	0.842	0.000
GIKT	0.878	0.878	0.000	0.860	0.860	0.000
SKT	0.625	0.624	-0.000	0.800	0.800	0.000
DyGKT	0.752	0.749	-0.003	0.813	0.812	-0.001
DGEKT	0.841	0.841	-0.000	0.844	0.844	-0.001

Table 25: Train-only vs. full-log graph ablation: Synthetic C2.

Model	AUC to	AUC fl	ΔAUC	ACC to	ACC fl	ΔACC
GKT	0.873	0.873	-0.001	0.808	0.807	-0.001
GIKT	0.875	0.874	-0.000	0.809	0.809	0.000
SKT	0.708	0.708	0.000	0.722	0.723	0.001
DyGKT	0.710	0.710	0.000	0.726	0.726	-0.000
DGEKT	0.710	0.711	0.001	0.724	0.725	0.001

Table 26: Train-only vs. full-log graph ablation: Synthetic C5.

Model	AUC to	AUC fl	ΔAUC	ACC to	ACC fl	ΔACC
GKT	0.810	0.812	0.002	0.738	0.738	0.000
GIKT	0.820	0.820	-0.000	0.749	0.747	-0.002
SKT	0.810	0.809	-0.001	0.740	0.739	-0.001
DyGKT	0.826	0.826	0.000	0.751	0.751	0.001
DGEKT	0.830	0.830	0.000	0.756	0.756	-0.001

Table 27: Paired comparisons against *simpleKT* on public benchmarks (three-fold learner CV). Δ is mean AUC difference (model minus *simpleKT*); fold-level magnitudes in the main baseline-CV table are the primary evidence. p_t and p_W are reported for transparency only (underpowered at n=3 folds).

Dataset	Model	ΔAUC	p_t	p_W
ASSISTments 2012	GKT	−0.0069	<0.001	0.25
	GIKT	−0.0005	0.001	0.25
	SKT	−0.0154	<0.001	0.25
	DyGKT	−0.0160	<0.001	0.25
	DGEKT	−0.0079	<0.001	0.25
XES3G5M	GKT	−0.0411	<0.001	0.25
	GIKT	+0.0031	<0.001	0.25
	SKT	−0.2501	<0.001	0.25
	DyGKT	−0.1223	<0.001	0.25
	DGEKT	−0.0334	<0.001	0.25
Junyi Academy	GIKT	−0.0021	0.445	1.00
	SKT	−0.0055	0.031	0.25
	DyGKT	−0.0066	0.100	0.25
	DGEKT	−0.0034	0.267	0.25

Table 28: Sequence-autocorrelation diagnostics on the three public benchmarks. *KC rep.* and *Item rep.* are the fractions of consecutive within-learner interactions that repeat the previous KC or item; *Persist. acc.* is $P(y_t{=}y_{t-1})$; *Lag-1* ρ is the lag-one label autocorrelation; *Prev-AUC* is the AUC of the trivial predictor $\hat{y}_t{=}y_{t-1}$.

Dataset	KC rep.	Item rep.	Persist. acc.	Lag-1 ρ	Prev-AUC
Junyi Academy	0.755	0.755	0.786	0.244	0.622
ASSISTments 2012	0.662	0.000	0.664	0.207	0.604
XES3G5M	0.211	0.145	0.786	0.342	0.671

Table 29: Ground-truth cross-validation on Junyi: edge/direction/reachability metrics and node Jaccard across top-K cutoffs. The main manuscript table reports edge precision/recall, direction agreement, and reachability F1 only.

K	Edge F1	Dir. agr.	Rch. F1	Node Jaccard
200	0.071	0.346	0.008	0.290
500	0.109	0.225	0.021	0.453
800	0.150	0.280	0.062	0.572
|expert|=1131	0.183	0.261	0.104	0.632
1200	0.185	0.262	0.126	0.641
1800	0.188	0.278	0.260	0.644
2500	0.187	0.285	0.339	0.610
5000 (cap)	0.187	0.285	0.339	0.610

Table 30: Per-stratum cold-start KC diagnostic with *simpleKT* on fold 0 (Section 3.7). The n column counts validation+test held-out interactions whose KC falls in the stratum train-frequency bin; *out of range* marks held-out KCs outside the four canonical bins. Stratum mass varies by dataset split; under default bins Synthetic C2/C5 route all held-out interactions into the hot stratum (> 500 train-fold hits per knowledge component).

Dataset	Stratum	n	AUC	ACC	NLL
Junyi Academy	very_cold	11	–	1.000	0.144
	cold	43	–	0.674	0.521
	warm	2,320	0.834	0.769	0.491
	hot	6,192,843	0.982	0.946	0.132
	out of range	5	–	0.800	0.474
ASSIST'12	very_cold	49	0.608	0.755	0.631
	cold	707	0.769	0.717	0.565
	warm	9,017	0.765	0.687	0.587
	hot	807,178	0.693	0.704	0.574
XES3G5M	very_cold	161	0.810	0.814	0.424
	cold	2,592	0.798	0.794	0.453
	warm	17,655	0.790	0.805	0.431
	hot	1,902,424	0.717	0.796	0.467
	out of range	6	0.625	0.667	0.638
Synthetic C2	hot	60,000	0.624	0.683	0.607
Synthetic C5	hot	60,000	0.630	0.611	0.651

Table 31: Cold-start comparison of sequence-only (No-Graph) and graph-aware (Graph) KT models. Entries are three-fold mean ± std AUC by KC train-frequency stratum; $\Delta_{\max}$ is the best Graph AUC minus the best No-Graph AUC in the same dataset/stratum. Cells marked "—" are suppressed when stratum AUC is undefined or unreliable (fewer than ten discordant label pairs, or identical AUC across all models within 10^{-6}).

		No-Graph		**Graph**			
Dataset	Stratum	DKT	SimpleKT	GKT	GIKT	DGEKT	$\Delta_{\max}$
assist2012	very_cold	0.723 ± 0.085	0.724 ± 0.100	0.695 ± 0.100	0.726 ± 0.102	0.724 ± 0.100	+0.002 ± 0.143
	cold	0.769 ± 0.004	0.772 ± 0.003	0.750 ± 0.013	0.772 ± 0.003	0.772 ± 0.003	+0.000 ± 0.013
	warm	0.751 ± 0.016	0.756 ± 0.017	0.724 ± 0.015	0.757 ± 0.017	0.756 ± 0.017	+0.001 ± 0.024
	hot	0.687 ± 0.002	0.696 ± 0.002	0.601 ± 0.001	0.691 ± 0.002	0.687 ± 0.002	-0.005 ± 0.003
junyi	very_cold	—	—	—	—	—	-
	cold	—	—	—	—	—	-
	warm	0.780 ± 0.023	0.846 ± 0.016	—	0.851 ± 0.019	0.824 ± 0.019	+0.005 ± 0.029
	hot	0.982 ± 0.000	0.982 ± 0.000	—	0.982 ± 0.000	0.981 ± 0.000	+0.000 ± 0.000
synthetic_c2	hot	0.619 ± 0.002	0.626 ± 0.002	0.554 ± 0.000	0.634 ± 0.002	0.634 ± 0.002	+0.008 ± 0.003
synthetic_c5	hot	0.628 ± 0.003	0.630 ± 0.003	0.524 ± 0.003	0.633 ± 0.002	0.633 ± 0.002	+0.002 ± 0.005
xes3g5m	very_cold	0.820 ± 0.012	0.817 ± 0.013	0.826 ± 0.004	0.818 ± 0.013	0.818 ± 0.013	+0.005 ± 0.018
	cold	0.802 ± 0.006	0.804 ± 0.007	0.793 ± 0.007	0.804 ± 0.008	0.803 ± 0.008	-0.000 ± 0.010
	warm	0.783 ± 0.003	0.787 ± 0.003	0.745 ± 0.004	0.780 ± 0.003	0.776 ± 0.003	-0.007 ± 0.005
	hot	0.708 ± 0.000	0.717 ± 0.000	0.606 ± 0.000	0.709 ± 0.000	0.703 ± 0.000	-0.009 ± 0.000

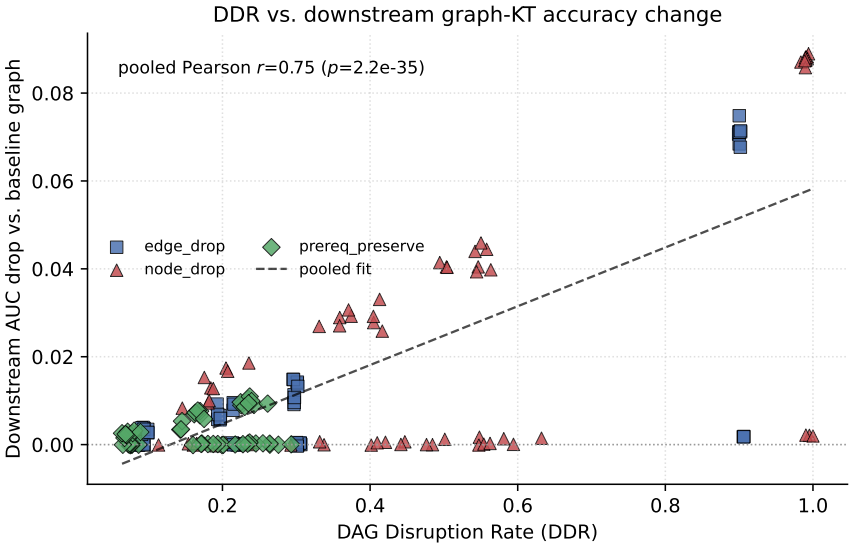
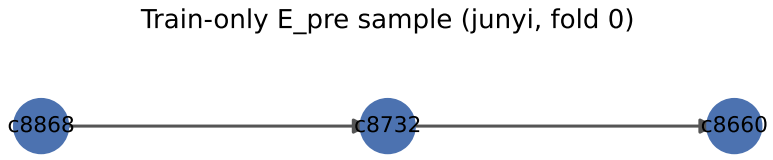
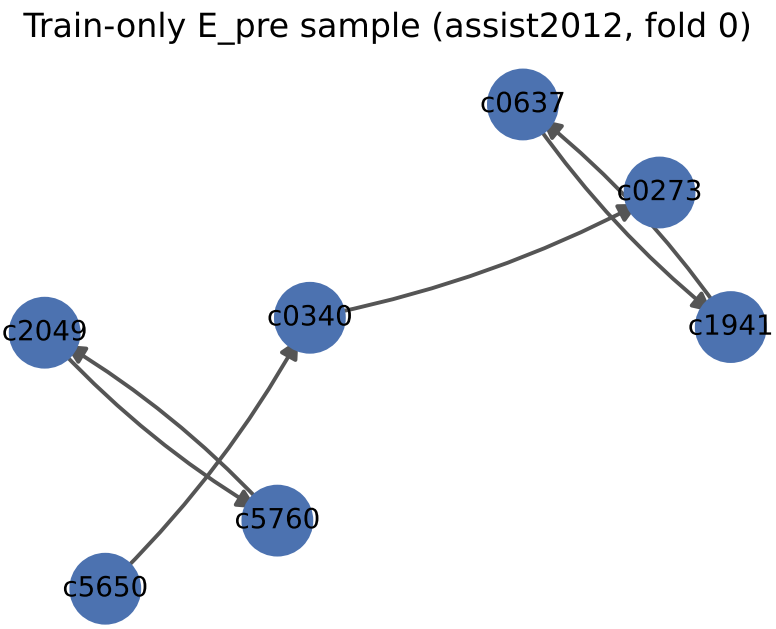


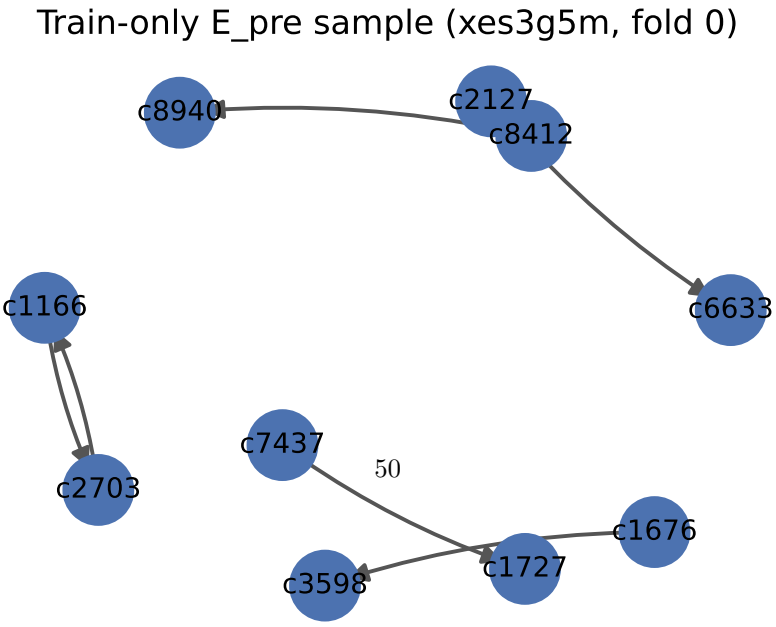
Fig. 8: Supplementary Figure S3. Downstream graph-KT sensitivity to prerequisite-graph disruption on ASSISTments 2012 and XES3G5M (DGEKT and GKT cells). Each point is one fold/operator/strength run; the vertical axis is AUC drop relative to the unperturbed train-only graph. The burned-in annotation $r\approx0.75$ is the *global pooled* Pearson correlation across both datasets and both models (n=186), not the XES3G5M/GKT-only correlation ($r\approx0.99$ with anchors, n=66; $r\approx0.95$ on the $p\leq0.3$ core, n=54).



(a) Junyi Academy

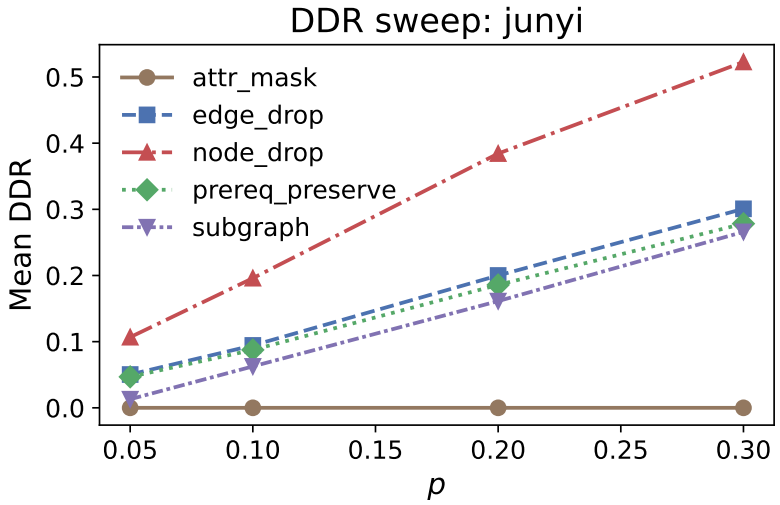


(b) ASSISTments 2012

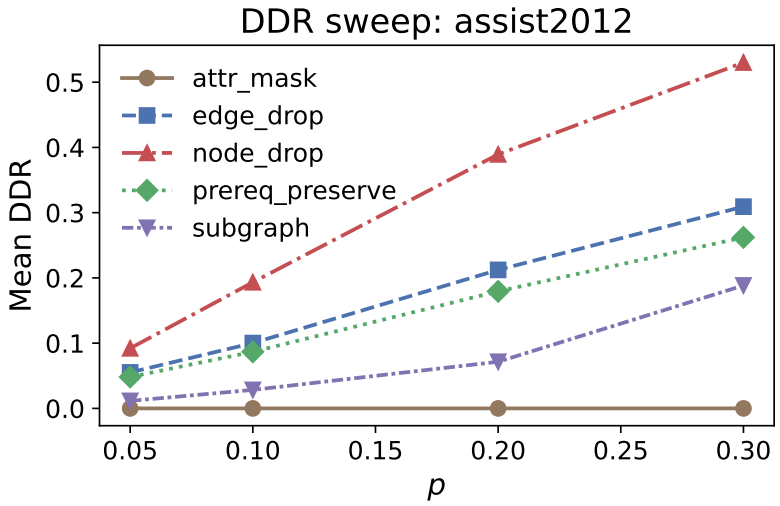


(c) XES3G5M

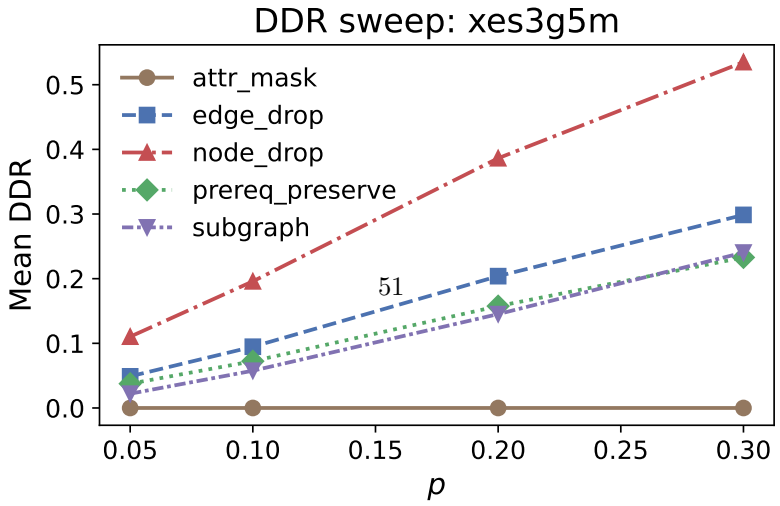
Fig. 9: Supplementary Figure S1. High-weight samples from train-only E_{pre} after DAG audit (fold 0 exports). Directed prerequisite edges are drawn as arrows. The main text shows the Junyi panel only.



(a) Junyi Academy



(b) ASSISTments 2012



(c) XES3G5M

Fig. 10: Supplementary Figure S2. DDR versus perturbation strength p for five augmentation families across three benchmarks. The main text shows the Junyi panel only.

Table 32: Training configurations and hyperparameters (Table S15)

Model	Codebase	Graph	Epochs	Batch	ES	Runtime	HW
BKT	`scipy` L-BFGS	—	30	64	N/A	<0.1 h	CPU
DKT	`pyKT` stock	—	30	64	5	∼0.5–3 h	RTX 3090
simpleKT	`pyKT` stock	—	30	64	5	∼0.5–3 h	RTX 3090
AKT	`pyKT` stock	—	30	64	5	∼0.5–3 h	RTX 3090
GKT	`pyKT` stock	$E_{\text{pre}}+E_{\text{sim}}$	10	4	5	∼2–4 h	RTX 3090
GIKT	native	$E_{\text{pre}}+E_{\text{sim}}$	10	8	5	∼0.5–3 h	RTX 3090
SKT	native	$E_{\text{pre}}+E_{\text{sim}}$	10	16	5	∼0.5–3 h	RTX 3090
DyGKT	native	$E_{\text{pre}}+E_{\text{sim}}$	10	16	5	∼0.5–3 h	RTX 3090
DGEKT	native	$E_{\text{pre}}+E_{\text{sim}}$	10	16	5	∼0.5–3 h	RTX 3090

Primary GKT/*simpleKT* budgets are author-attested (commit `619c02cf`): GKT max. 10 epochs, batch 4; *simpleKT* max. 30 epochs, batch 64. HW = RTX 3090 denotes one NVIDIA GeForce RTX 3090 (24 GB); not A100/V100. Runtime is approximate wall-clock on that GPU (CPU for BKT), reported in hours. GKT ∼2–4 h corresponds to an observed ∼0.33 h/epoch (∼20 min/epoch) on XES3G5M-scale folds at the 10-epoch cap with early stopping; other GPU rows are coarse order-of-magnitude estimates. Table S21: exploratory seed-42 three-fold GKT check (max. 30 epochs, batch 32); not compute-matched / not epoch-only. Legacy unpaired GKT30 seeds 17/1234 excluded.

M Training-configuration disclosure

Table S15 (Table 32) reports the training configurations and model-specific hyperparameters used across the `pyKT` and native baselines. Settings shared by all models are kept out of the table: learning rate 0.001 (Adam), zero hyperparameter-search budget, base split seed 42 with fold seeds 42–44, identical processed splits under both train-only and full-log graph constructions, and checkpoint selection by best validation AUC with early stopping (BKT is fit by L-BFGS on NLL without early stopping). Epoch caps and batch sizes differ across families (maximum 30 epochs for sequence checkpoints vs. maximum 10 for graph backbones in the primary release; primary GKT batch 4, *simpleKT* batch 64), reflecting per-epoch cost. The exploratory extended-training check is detailed in Section 5.3 and Table 34. "Native" denotes our re-implementations in `src/models`; graph models consume the exported E_{pre} and, where available, E_{sim} edges.

N ΔAUC intervals for primary pairs

Table S16 (Table 33) reports the mean AUC difference (ΔAUC) between the primary model pairs on XES3G5M under train-only graph construction. The released bundle reports paired-t 95% intervals over three learner-disjoint folds (default; matches fold-level ΔAUC). Optional row-level learner bootstrap: `scripts/bootstrap_auc_ci.py--learner-bootstrap`. The GKT deficit under the released 10-epoch budget (-0.041; CI $[-0.044, -0.038]$) is more than an order of magnitude larger than the interval half-width, so the model-specific conclusion does not hinge on fragile three-fold t-tests. Extended-training sensitivity for GKT is consolidated in Section 5.3 (Table 34). Reproducers may optionally run `python scripts/bootstrap_auc_ci.py --learner-bootstrap` on exported prediction parquets for row-level learner resampling (slower; fold-level paired-t is the default).

Table 33: ΔAUC intervals for primary pairs, XES3G5M, train-only graphs (Table S16)

Model Pair	ΔAUC	95% CI (paired t, 3 folds)	n_{learners}	n_{rows}
GKT vs *simpleKT*	−0.041	[−0.044, −0.038]	—	—
GIKT vs *simpleKT*	+0.003	[+0.003, +0.003]	—	—

O Targeted three-fold extended-training configuration sensitivity

Table S21 (Table 34) reports a targeted three-fold extended-training configuration-sensitivity check on XES3G5M using experiment seed 42 and folds 0–2. Primary GKT artefacts use a maximum of 10 epochs and batch size 4; the extended-training configuration uses a maximum of 30 epochs and batch size 32 (hidden dimension and maximum sequence length also differ from primary). Fold AUCs are unchanged historical artefacts; legacy unpaired GKT30 artefacts at seeds 17 and 1234 are excluded from submission. This check is exploratory and does not isolate the effect of epochs.

Table 34: Targeted three-fold extended-training configuration-sensitivity check on XES3G5M using experiment seed 42. The primary GKT configuration used a maximum of 10 epochs and batch size 4, whereas the extended-training configuration used a maximum of 30 epochs and batch size 32. The observed mean AUC was 0.003451 higher under the extended-training configuration; however, the approximate 95% confidence interval [−0.004, 0.011] included zero. Because maximum epochs, batch size, hidden dimension, and sequence length all changed, this exploratory comparison does not isolate the effect of epochs.

Fold	GKT primary AUC	Extended-training GKT AUC	ΔAUC
0	0.834557	0.840181	+0.005624
1	0.833752	0.838300	+0.004547
2	0.832624	0.832804	+0.000181
Mean	0.833644	0.837095	+0.003451

P Controlled Leak Injection

Table S17 (Table 35) probes the sensitivity of the audit metrics by deliberately injecting 5% and 20% of test-fold interactions into the train-only prerequisite builder on XES3G5M (fold 0). Edge counts $|E_{\text{pre}}|$ are *candidate* edges before DAG pruning (the main text DAG audit reports 701 retained edges after pruning on the clean fold). Candidate-edge count and TBMR_f increase monotonically with the injection rate,

while $\mathrm{ECR}_f^{\mathrm{flag}}$ remains 0 (learner-id sets stay disjoint) and $\mathrm{ECR}_f^{\mathrm{overlap}}$ is already saturated at 1.0 on this transition-dense corpus. Flag-level checks alone therefore miss this contamination channel, supporting the tiered audit design in the main text.

Table 35: Controlled Leak Injection, XES3G5M fold 0. $|E_{\mathrm{pre}}|$ counts candidate edges before DAG pruning. (Table S17)

Injection rate	$\lvert E_{\mathrm{pre}}\rvert$	$\mathrm{ECR}^{\mathrm{overlap}}$	$\mathrm{ECR}^{\mathrm{flag}}$	TBMR
0%	1162	1.000	0.000	0.754
5%	1378	1.000	0.000	0.776
20%	1417	1.000	0.000	0.779

Table S18 (Table 36) reports fold-0 test AUC after retraining *simpleKT*, GKT, and GIKT on the clean and injected graphs under Table S15 budgets (GPU-verified batch run, Server A, 2026-07-23). Headline AUC does not track the structural audit monotonically: at 20% injection, *simpleKT* and GIKT move by $\leq$0.0004 AUC while GKT dips by $\approx$0.010, despite a +21.9% candidate-edge increase in Table S17. We therefore treat S18 as evidence that metric-tier reporting must precede AUC-only leakage conclusions, not as proof that contamination always lowers or raises accuracy.

Table 36: Downstream test AUC under controlled leak injection, XES3G5M fold 0 (Table S18). Models retrained under reduced graph-backbone budgets (Table S15: 10 epochs, smaller batches); qualitative decoupling (builder mass vs. flat AUC) is the intended reading, not cross-table absolute AUC ranking.

Model	**0%**	**5%**	**20%**
simpleKT	0.8744	0.8748	0.8748
GKT	0.8346	0.8255	0.8243
GIKT	0.8776	0.8778	0.8778

Q Exploratory ANOVA summaries

The main text reports fold-level means $\pm$ std throughout Tables 8, 9, 5, 11, and 14, with Table S16 ΔAUC intervals (Table S16) as the primary inferential evidence for the XES3G5M ranking observation. For reviewer-requested completeness, Tables 37–38

Table 37: Exploratory two-way ANOVA on fold-level AUC (n=3 folds per dataset–model cell; public benchmarks only, train-only graphs; Table S19). Partial $\eta^2 = \mathrm{SS}_{\mathrm{effect}}/(\mathrm{SS}_{\mathrm{effect}} + \mathrm{SS}_{\mathrm{error}})$. With only three folds per cell, p-values are underpowered; fold-level ΔAUC magnitudes and Table S16 intervals are the primary evidence.

Source	SS	df	MS	p	η_p^2
Dataset	0.3365	2	0.1682	<0.001	1.000
Backbone	0.0701	5	0.0140	<0.001	0.998
Dataset × Backbone	0.0968	10	0.0097	<0.001	0.999
Residual	0.0001	34	0.0000	—	—

Table 38: ANOVA on GKT downstream AUC under graph perturbation (GKT multi-seed sweep on XES3G5M (seeds 42/17/1234) plus ASSISTments seed-42 folds; core $p \leq 0.3$ grid (anchors p=0.90 excluded); Table S20). Dependent variable: test AUC after retraining on the perturbed graph. Partial η^2 as above. This analysis supports the DDR→downstream findings in Section 4.7.

Source	SS	df	MS	p	η_p^2
Dataset	0.4082	1	0.4082	<0.001	0.997
Operator	0.0062	2	0.0031	<0.001	0.817
Operator × strength p	0.0032	6	0.0005	<0.001	0.699
Dataset × Operator	0.0019	2	0.0009	<0.001	0.575
Residual	0.0014	96	0.0000	—	—

(Tables S19–S20) add exploratory ANOVA models on fold-level observations. Because each dataset–model cell contains only three folds, we interpret ANOVA p-values as descriptive rather than as proof of backbone dominance.

Acknowledgements. The authors thank the maintainers of the Junyi Academy, ASSISTments, and XES3G5M datasets and of the `pyKT` benchmark library, whose public releases made this study possible.

Declaration of generative AI-assisted work

During the preparation of this manuscript, the authors used generative AI tools, including ChatGPT and Cursor, to assist with language refinement, internal-consistency checking, and manuscript-quality auditing. All AI-assisted outputs were reviewed by the authors against the available manuscript sources, references, configurations, result artifacts, and audit records. The authors made all final scientific and editorial decisions and take full responsibility for the content. No generative AI tool was listed as an author.

Statements and Declarations

Funding. The authors declare that no funds, grants, or other support were received during the preparation of this manuscript.

Competing interests. The authors have no competing interests to declare that are relevant to the content of this article.

Ethics approval. This study uses only publicly available, de-identified knowledge-tracing benchmark datasets under the dataset-specific licences and terms of use listed under Data availability below. No human-subjects experiments were conducted and no ethics approval was required. No claims about real-world learning outcomes are made.

Consent to participate / Consent for publication. Not applicable; the study involves no identifiable individuals.

Data availability. All datasets analysed in this study are publicly available. Raw interaction files are downloaded separately by reproducers under each dataset's licence or terms of use. The repository ships preprocessing code, configurations, and derived paper tables and figures; fold-specific graph exports (`e\_pre\_train\_only.csv`) and audit logs are regenerated by the documented pipeline. Reproducers regenerate graphs with `python-msrc.graph_builder` after preprocessing.

- **Junyi Academy:** PSLC DataShop dataset 1198 (`https://pslcdatashop.web.cmu.edu/DatasetInfo?datasetId=1198`; interaction log `junyi_ProblemLog_original.csv`; accessed 2026-06-08), under PSLC DataShop terms of use.
- **ASSISTments 2012:** 2012–13 school-year release with affect predictions (`https://sites.google.com/site/assistmentsdata/datasets/2012-13-school-data-with-affect`), under the ASSISTments Data terms of use (`https://sites.google.com/site/assistmentsdata/termsofuseforusingdata`).
- **XES3G5M:** NeurIPS 2023 Datasets and Benchmarks release (`https://github.com/ai4ed/XES3G5M`; MIT Licence).
- **Synthetic C2/C5:** companion tiny-graph sanity logs. Their configurations and derived audit artefacts are included in the repository; the raw interaction matrices are not redistributed.

Code availability. Source code, configurations, pipeline scripts, and result artefacts are available at `https://github.com/edu-risk-lab/leakage-controlled-kt-audit.git`. Baseline KT training and evaluation use the open-source `pyKT` toolkit (`https://github.com/pykt-team/pykt-toolkit`; MIT Licence), cited as [17]; other scientific Python/PyTorch dependencies are used under their respective upstream open-source licences. Paper-facing tables with fold-level mean ± std and inferential summaries are regenerated by `scripts/generate_phase_c_tables.py` (invoked from `scripts/generate_paper_artifacts.py`); ΔAUC intervals (Table S16) use `scripts/bootstrap_auc_ci.py` (paired-t over three folds by default; `--learner-bootstrap` for optional row-level resampling on parquets). Supplementary tables and figures are included in the Appendix of this document.

Appendix. Supplementary tables and figures are merged into the Appendix (overview at Section 6). Tables S1–S5: per-dataset baseline diagnostics; S6–S10: train-only versus full-log ablation; S11: paired significance tests on public benchmarks; S12–S13:

cold-start strata; S14: ground-truth metrics including node Jaccard; S15: training-configuration disclosure; S16: ΔAUC intervals for primary pairs; Tables S17–S18 report controlled leak-injection metrics and downstream AUC on injected graphs. Table S21 reports the targeted seed-42 three-fold extended-training configuration-sensitivity check. Tables S19–S20 report ANOVA summaries for baselines and the GKT multi-seed DDR→downstream sweep (Section Q in the Appendix). Section S3: sequence-autocorrelation statistics; Sections S4–S5: augmentation-operator definitions and ground-truth disagreement examples. Figures S1–S2: multi-benchmark train-only E_{pre} samples and DDR curves; Figure S3: DDR downstream scatter plot; Figures S6–S7: cold-start panels and Junyi ground-truth PR curve. Cold-start and GT panels are generated from fold 0 exports.

Author contributions. T. Dao Minh conceived the protocol, implemented the leakage-controlled pipeline, ran the experiments, and drafted the manuscript. K.-T. Nguyen and D. Nguyen Tien contributed to the experimental implementation, data preprocessing, and result analysis. Q. K. Ngo contributed to manuscript revision and technical review. V.-H. Nguyen and L. Hoang Son supervised the work, advised on the experimental design, and revised the manuscript. All authors read and approved the final manuscript.

References

[1] Corbett, A.T., Anderson, J.R.: Knowledge tracing: Modeling the acquisition of procedural knowledge. User Modeling and User-Adapted Interaction **4**(4), 253–278 (1995) https://doi.org/10.1007/BF01099821

[2] Piech, C., Bassen, J., Huang, J., Ganguli, S., Sahami, M., Guibas, L.J., Sohl-Dickstein, J.: Deep knowledge tracing. In: Advances in Neural Information Processing Systems (NeurIPS), pp. 505–513 (2015). https://doi.org/10.48550/arXiv.1506.05908

[3] Zhang, J., Shi, X., King, I., Yeung, D.-Y.: Dynamic key-value memory networks for knowledge tracing. In: Proceedings of the 26th International Conference on World Wide Web (WWW), pp. 765–774 (2017). https://doi.org/10.1145/3038912.3052580

[4] Ghosh, A., Heffernan, N.T., Lan, A.S.: Context-aware attentive knowledge tracing. In: Proceedings of the 26th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD), pp. 2330–2339 (2020). https://doi.org/10.1145/3394486.3403282

[5] Liu, Z., Liu, Q., Chen, J., Huang, S., Luo, W.: simpleKT: A simple but tough-to-beat baseline for knowledge tracing. In: International Conference on Learning Representations (ICLR) (2023). https://doi.org/10.48550/arXiv.2302.06881

[6] Pandey, S., Karypis, G.: A self-attentive model for knowledge tracing. In: Proceedings of the 12th International Conference on Educational Data Mining (EDM),

pp. 384–389 (2019). https://doi.org/10.48550/arXiv.1907.06837

[7] Shen, S., Liu, Q., Chen, E., Wu, H., Huang, Z., Zhao, W., Su, Y., Ma, H., Wang, S.: Convolutional knowledge tracing: Modeling individualization in student learning process. In: Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval, pp. 1857–1860 (2020). https://doi.org/10.1145/3397271.3401288

[8] Nakagawa, H., Iwasawa, Y., Matsuo, Y.: Graph-based knowledge tracing: Modeling student proficiency using graph neural network. In: IEEE/WIC/ACM International Conference on Web Intelligence (WI), pp. 156–163 (2019). https://doi.org/10.1145/3350546.3352513

[9] Tong, S., Liu, Q., Huang, W., Huang, Z., Chen, E., Liu, C., Ma, H., Wang, S.: Structure-based knowledge tracing: An influence propagation view. In: IEEE International Conference on Data Mining (ICDM), pp. 541–550 (2020). https://doi.org/10.1109/ICDM50108.2020.00063

[10] Yang, Y., Shen, J., Qu, Y., Liu, Y., Wang, K., Zhu, Y., Zhang, W., Yu, Y.: GIKT: A graph-based interaction model for knowledge tracing. In: Machine Learning and Knowledge Discovery in Databases (ECML-PKDD 2020). LNCS, vol. 12457, pp. 299–315 (2021). https://doi.org/10.1007/978-3-030-67658-2_18

[11] Cheng, K., Peng, L., Wang, P., Ye, J., Sun, L., Du, B.: DyGKT: Dynamic graph learning for knowledge tracing. In: Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD), pp. 409–420 (2024). https://doi.org/10.1145/3637528.3671773

[12] Cui, C., Yao, Y., Zhang, C., Ma, H., Ma, Y., Ren, Z., Zhang, C., Ko, J.: DGEKT: A dual graph ensemble learning method for knowledge tracing. ACM Transactions on Information Systems **42**(3), 1–24 (2024) https://doi.org/10.1145/3638350

[13] Tong, H., Wang, Z., Liu, Q., Zhou, Y., Han, W.: HGKT: Introducing problem schema with hierarchical exercise graph for knowledge tracing. In: Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval, pp. 405–415 (2022). https://doi.org/10.1145/3477495.3532004

[14] Kaufman, S., Rosset, S., Perlich, C., Stitelman, O.: Leakage in data mining: Formulation, detection, and avoidance. ACM Transactions on Knowledge Discovery from Data **6**(4), 1–21 (2012) https://doi.org/10.1145/2382577.2382579

[15] Kapoor, S., Narayanan, A.: Leakage and the reproducibility crisis in machine-learning-based science. Patterns **4**(9) (2023) https://doi.org/10.1016/j.patter.2023.100804

[16] Schein, A.I., Popescul, A., Ungar, L.H., Pennock, D.M.: Methods and metrics

for cold-start recommendations. In: Proceedings of the 25th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval, pp. 253–260 (2002). https://doi.org/10.1145/564376.564421

[17] Liu, Z., Liu, Q., Chen, J., Huang, S., Tang, J., Luo, W.: pyKT: A Python library to benchmark deep learning based knowledge tracing models. In: Advances in Neural Information Processing Systems (NeurIPS), vol. 35, pp. 18542–18555 (2022). https://doi.org/10.52202/068431-1347

[18] Badran, Y., Preisach, C.: Addressing label leakage in knowledge tracing models. arXiv preprint arXiv:2403.15304 (2024) https://doi.org/10.48550/arXiv.2403.15304

[19] You, Y., Chen, T., Sui, Y., Chen, T., Wang, Z., Shen, Y.: Graph contrastive learning with augmentations. In: Advances in Neural Information Processing Systems (NeurIPS), vol. 33, pp. 5812–5823 (2020). https://doi.org/10.48550/arXiv.2010.13902

[20] Zhu, Y., Xu, Y., Yu, F., Liu, Q., Wu, S., Wang, L.: Graph contrastive learning with adaptive augmentation. In: Proceedings of the Web Conference (WWW), pp. 2069–2080 (2021). https://doi.org/10.1145/3442381.3449802

[21] Heffernan, N.T., Heffernan, C.L.: The ASSISTments ecosystem: Building a platform that brings scientists and teachers together for minimally invasive research on human learning and teaching. International Journal of Artificial Intelligence in Education **24**(4), 470–497 (2014) https://doi.org/10.1007/s40593-014-0024-x

[22] Junyi Academy: Junyi Academy Online Learning Activity Dataset. PSLC DataShop. Accessed 2026-06-08 (2015). https://pslcdatashop.web.cmu.edu/DatasetInfo?datasetId=1198

[23] Choi, Y., Lee, Y., Shin, D., Cho, J., Park, S., Lee, S., Baek, J., Bae, C., Kim, B., Heo, J.: EdNet: A large-scale hierarchical dataset in education. In: International Conference on Artificial Intelligence in Education (AIED), pp. 69–73 (2020). https://doi.org/10.1007/978-3-030-52240-7_13

[24] Liu, Z., Liu, Q., Guo, T., Chen, J., Huang, S., Zhao, X., Tang, J., Luo, W., Weng, J.: XES3G5M: A knowledge tracing benchmark dataset with auxiliary information. In: Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track, vol. 36, pp. 32958–32970 (2023). https://doi.org/10.52202/075280-1429

[25] Chang, H.-S., Hsu, H.-J., Chen, K.-T.: Modeling exercise relationships in e-learning: A unified approach. In: Proceedings of the 8th International Conference on Educational Data Mining (EDM), pp. 532–535 (2015). `http://www.educationaldatamining.org/EDM2015/proceedings/short532-535.pdf`

[26] Abdelrahman, G., Wang, Q., Nunes, B.P.: Knowledge tracing: A survey. ACM Computing Surveys **55**(11), 1–37 (2023) https://doi.org/10.1145/3569576

[27] Lee, W., Chun, J., Lee, Y., Park, K., Park, S.: Contrastive learning for knowledge tracing. In: Proceedings of the ACM Web Conference (WWW), pp. 2330–2338 (2022). https://doi.org/10.1145/3485447.3512105

[28] Pan, L., Li, C., Li, J., Tang, J.: Prerequisite relation learning for concepts in MOOCs. In: Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (ACL), pp. 1447–1456 (2017). https://doi.org/10.18653/v1/P17-1133

[29] Bai, Y., Liu, Z., Guo, T., Hou, M., Xiao, K.: Prerequisite relation learning: A survey and outlook. ACM Computing Surveys **57**(9), 1–36 (2025) https://doi.org/10.1145/3733593

[30] Kipf, T.N., Welling, M.: Semi-supervised classification with graph convolutional networks. In: International Conference on Learning Representations (ICLR) (2017). https://doi.org/10.48550/arXiv.1609.02907

[31] Hamilton, W.L., Ying, R., Leskovec, J.: Inductive representation learning on large graphs. In: Advances in Neural Information Processing Systems (NeurIPS), pp. 1024–1034 (2017). https://doi.org/10.48550/arXiv.1706.02216

[32] Johnson, D.B.: Finding all the elementary circuits of a directed graph. SIAM Journal on Computing **4**(1), 77–84 (1975) https://doi.org/10.1137/0204007